European Journal of Operational Research 336 (2027) 456–475

Contents lists available at ScienceDirect

European Journal of Operational Research

journal homepage: www.elsevier.com/locate/eor

Discrete Optimization

Resource-constrained parallel machine scheduling with setups, job release/due time, and resource transition limit

Maziyar Khadivi [a], Kevin Tierney [b], Mostafa Abbasi [a], Todd Charter [c], Homayoun Najjaran [a,c,*]

[a] Department of Mechanical Engineering, University of Victoria, Victoria, British Columbia, Canada
[b] Department of Business Decisions and Analytics, University of Vienna, Vienna, Austria
[c] Department of Electrical and Computer Engineering, University of Victoria, Victoria, British Columbia, Canada

ARTICLE INFO

Keywords:
Scheduling
Resource-constrained parallel machine
Mixed-integer linear programming
Biased random-key genetic algorithm
Metaheuristics

ABSTRACT

The unrelated parallel machines scheduling problem with setup times and additional resources (UPMSR) is common in the manufacturing and service sectors, requiring the optimal assignment of jobs to machines supervised by limited additional resources. This paper investigates how to efficiently solve the UPMSR considering manufacturing constraints such as job release times, due dates, machine eligibility, and resource transition limits per machine. Previous research has not addressed this combination of real-world industrial constraints, leaving limitations on practical and scalable scheduling algorithms for the UPMSR. To address this gap, we develop a mixed integer linear programming (MILP) model and introduce a novel biased random key genetic algorithm (BRKGA) approach. We demonstrate the scalability of BRKGA compared to the MILP model on synthetic datasets based on a real-world case study of a food processing plant. Furthermore, we demonstrate that BRKGA either outperforms or matches state-of-the-art metaheuristic and exact methods on benchmark datasets from the literature. This research provides a broadly adaptable, scalable, and license-free scheduling software tool, enabling practitioners and researchers to tackle similar resource-constrained parallel machine scheduling problems effectively.

1. Introduction

The area of machine scheduling consists of many diverse problem types, each tailored to model how jobs are processed by machines within a specific environment. The parallel machine scheduling problem (PMSP) in particular is one of the most studied machine scheduling problems in the literature (Edis et al., 2013). In the manufacturing domain, PMSP is suitable to model high output production shops or the most critical stage in certain production processes, for instance, the assembly stage in laptop manufacturing (Liang et al., 2022), packing hall operations in food processing plants (Basso & Varas, 2017), or photolithography machines in semiconductor manufacturing (Chen et al., 2022).

The PMSP involves the allocation of n independent jobs across m machines that perform tasks in parallel, and possibly at different speeds (Fanjul-Peyro et al., 2019). Additionally, machines may require cleaning, color preparation, or adjustment prior to processing the next job. These activities, referred to as setup times, makes the sequencing of jobs on each machine essential. In many real-world settings, auxiliary resources such as machine operators, dies, tools, pallets, and industrial robots are required during machine setup, job processing, or both (Afzalirad & Rezaeian, 2016). These resources are limited in availability, introducing an additional layer of complexity to the PMSP. Optimizing job allocation, sequencing, and timing while respecting resource constraints is inherently tied to the key performance indicators (KPIs) pursued by the manufacturing enterprise, including makespan, tardiness, production cost/time, and order fulfillment.

We address one of the complex forms of resource-constrained parallel machine scheduling, namely, the unrelated parallel machine scheduling problem with setup times and additional resources required during both job processing and machine setups (UPMSR-PS). Processing times vary by job-machine pair, and setup times depend on both the machine and job sequence. A single resource with limited quantity is available and each machine requires the assignment of one unit of the resource during setup and job processing. Motivated by a food processing plant case study,

* Corresponding author at: Department of Mechanical Engineering, University of Victoria, Victoria, British Columbia, Canada.
E-mail addresses: mazy1996@uvic.ca (M. Khadivi), kevin.tierney@univie.ac.at (K. Tierney), abbasi@uvic.ca (M. Abbasi), toddch@uvic.ca (T. Charter), najjaran@uvic.ca (H. Najjaran).

https://doi.org/10.1016/j.ejor.2026.05.025
Received 24 July 2025; Accepted 16 May 2026
Available online 26 May 2026
0377-2217/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

our model incorporates job release and due dates, machine eligibility constraints, and a limit on how frequently resources can switch between machines. While some studies (e.g., Caselli et al., 2024) have considered time windows and eligibility in related problems, these aspects have not been specifically addressed in the context of the UPMSR. Moreover, prior work assumes unrestricted resource movement.

Frequent resource transitions (e.g., moving a tool or worker between machines) can significantly reduce production efficiency due to the time required for adjustments, cleaning, and restarting production lines. Several studies model such transitions by introducing a transfer time when a resource switches between machines (e.g., Chen et al., 2022; Ham, 2017). However, incorporating transfer time solely through the objective function does not strictly cap the number of transitions or production runs, as required in our case study and in other industrial settings (Barták & Čepek, 2006). Moreover, enforcing such a restriction would necessitate a separate objective that minimizes the number of transfers prior to optimizing production performance. Introducing this type of multi-objective (lexicographic) optimization substantially increases computational complexity. In contrast, our proposed approach allows transfer times to be incorporated if desired, while also imposing an adjustable upper bound on the number of transitions per machine by partitioning the time horizon into sub-slots. This modeling choice becomes particularly effective when transfer times are difficult to quantify, or when transition costs extend beyond time losses (e.g., increased wear and tear or labor inefficiencies).

In this paper, we propose an efficient mixed-integer linear programming (MILP) model and a biased random key genetic algorithm (BRKGA) for the UPMSR-PS with production time windows and resource transition limits. Benchmarking on synthetic datasets derived from the food processing plant case study shows that BRKGA matches MILP across all instance sizes, achieving small optimality gap with shorter average run time. For large-scale instances, the BRKGA outperforms the MILP model by consistently finding feasible solutions for a greater number of problems, demonstrating its superior scalability in practical settings. We further adapt the BRKGA decoder to compare it with the state-of-the-art greedy randomized adaptive search procedure (GRASP) metaheuristic (Lopez-Esteve et al., 2023) and exact methods (Caselli et al., 2024), both designed for similar resource-constrained PMSPs (RPMSP). The results show that the BRKGA matches or exceeds these methods in solution quality and solution time. Overall, our BRKGA is particularly effective for RPMSPs where (i) defining search neighborhoods is nontrivial, as highlighted by Avgerinos et al. (2023), or (ii) designing a two-phase constructive–repair heuristic, as adapted by Villa et al. (2018), Yepes-Borrero et al. (2021, 2020), and Lopez-Esteve et al. (2023), is difficult due to multiple interacting feasibility constraints. These two features characterize our problem setting and other constrained RPMSPs such as the work of Caselli et al. (2024) that have so far been addressed only via Benders-decomposition (BD) with degrading performance at scale.

The remainder of the paper is structured as follows. Section 2 reviews related work. Section 3 defines the problem, and provides an example. Section 4 and Section 5 present the MILP model and the BRKGA approach, respectively. Section 6 describes instance generation, MILP and BRKGA results on synthetic instances, BRKGA comparison with the state-of-the-art methods, and managerial insights from the case study. Section 7 concludes the paper and highlights future research directions.

2. Literature review

Most work on RPMSPs relies on exact methods, mainly MILP and constraint programming (CP). CP has proved effective for timing subproblems within decomposition schemes (Avgerinos et al., 2023; Fanjul-Peyro, 2020). It has also been applied to UPMSR instances with constraints similar to those in our problem, namely UPMSR-PS with production time windows and resource transition limits, though problem sizes remain limited to about 60 jobs (Yunusoglu & Topaloglu Yildiz, 2022). By contrast, MILP models have been shown to handle RPMSPs at larger scales with complex side constraints (Caselli et al., 2024).

To improve scalability, several studies propose decomposition-based exact algorithms for RPMSPs, typically splitting the problem into allocation, sequencing, and timing decisions and solving them in a master–subproblem framework (Avgerinos et al., 2023; Caselli et al., 2024; Fang et al., 2021; Fanjul-Peyro, 2020). These studies, except that of Caselli et al. (2024), consider RPMSPs without complex side constraints. The work presented by Caselli et al. (2024) combines a Benders-decomposition (BD) with a CP warm start to handle release, precedence, and contiguity constraints. Other authors have proposed math-heuristics to reduce the search space of MILPs (Fanjul-Peyro et al., 2017) or used heuristics to warm-start exact models (Caricato et al., 2021; Fleszar & Hindi, 2018; Heinz et al., 2022; Munoz et al., 2022). Although BD and hybrid heuristic–exact approaches can accelerate commercial solvers, they still suffer from infeasibility, large optimality gaps, or excessive run times on large-scale instances (Avgerinos et al., 2023; Lopez-Esteve et al., 2023).

To address the scalability limitation of exact methods, heuristic and metaheuristic approaches are widely used to solve RPMSPs. These studies cover various RPMSP variants (Table 1), typically using a two-stage solution framework: a constructive heuristic to generate an initial schedule, a repair phase to restore resource feasibility, and an improvement phase based on local search or metaheuristics (e.g., restricted local search, scatter search, GRASP) to improve solution quality (Lopez-Esteve et al., 2023; Villa et al., 2018; Yepes-Borrero et al., 2021, 2020). Other studies rely on heuristic rules (Kim & Lee, 2021; Zhang et al., 2021) or genetic algorithms (Chen et al., 2022; Li et al., 2024) to solve large-scale RPMSPs, but typically without complex side constraints. When precedence or release/due-date constraints are considered, time-window constraints are often relaxed (Li et al., 2024; Ruiz-Torres et al., 2007) or tested only on small instances (Afzalirad & Rezaeian, 2016; Al-Harkan & Qamhan, 2019; Tamaki et al., 1993). We refer the reader to Table 1 for a comprehensive overview of the RPMSP literature.

3. Problem definition and example

We study the UPMSR-PS that we define in Section 1. We assume preemption is not allowed, and there is no precedence relationship among jobs. Some jobs can only be processed by a subset of eligible machines. The resource is renewable and retained until the machine setup and processing of the current job is completed. The scheduling horizon comprises discrete time periods, such as weeks, and each time period is time-continuous, for instance, expressed in minutes. When a resource is transferred to a new machine, a transfer time is incurred before processing on that machine can begin. Resources are fully or partially available within each time period, with their availability defined by start and end work times. The release of a job is defined by a release period and a release time. The release period shows the time period in which the order is released, and the release time determines the time during that period the order is released. Similar definitions hold for the due period and due time.

To ensure high production efficiency, the number of resource transitions per machine within a given time period is restricted by a predefined upper limit. A resource transition refers to the event when a resource completes its task on one machine and moves to another machine to begin a new task. We further explain this concept with the following toy example. The objective function is to find an optimal schedule that minimizes the total production time consisting of processing and setup times under these operating constraints.

Table 1
Summary of previous works focused on solving RPMSP.

#	Authors (Year)	Machine env.	Res. type	Res. Trans.	Res. /task	Manufacturing aspects	Solution method	Max. problem size ($Job \times Machine \times Resource$)
1	Tamaki et al. (1993)	UPMSR	PS	UL	F	DD, ME	MH	(10 × 4 × 7)
2	Chen (2005)	UPMSR	PS	UL	F	–	H	(70 × 8 × 11)
3	Ruiz-Torres et al. (2007)	IPMR	P	F	V	DD	MILP/H	(200 × 10 × 30)
4	Celano et al. (2008)	UPMSR	S	UL	F	–	MH	(50 × 14 × 14)
5	Costa et al. (2013)	UPMSR	S	UL	F	–	MINLP/MH	(10 × 5 × 3)
6	Afzalirad and Rezaeian (2016)	UPMSR	P	UL	V	RD, ME, PC	MINLP/MH	(60 × 8 × 2)
7	Bitar et al. (2016)	UPMSR	P	UL	F	ME	MH	(200 × 8 × 20)
8	Ham (2017)	IPMSR	P	UL	F	RD	MILP	(200 × 30 × 200)
9	Fanjul-Peyro et al. (2017)	UPMR	P	UL	V	–	MILP/CP/Matheuristic	(30 × 6 × 30)
10	Fleszar and Hindi (2018)	UPMR	P	UL	V	–	H (MILP+CP)	(1000 × 15 × 75)
11	Villa et al. (2018)	UPMR	P	UL	V	–	H	(350 × 30 × 150)
12	Vallada et al. (2019)	UPMR	P	UL	V	–	MH	(350 × 30 × 150)
13	Al-Harkan and Qamhan (2019)	UPMSR	P	UL	V	RD	MILP/MH	(60 × 8 × 8)
14	Bektur and Saraç (2019)	UPMSR	S	UL	F	–	MILP/MH	(100 × 10 × 1)
15	Akyol Ozer and Sarac (2019)	IPMSR	PS	UL	F	ME	MH + MILP	(100 × 8 × 40)
16	Yepes-Borrero et al. (2020)	UPMSR	S	UL	V	–	MILP/MH	(250 × 30 × 4)
17	Fanjul-Peyro (2020)	UPMSR	PS	UL	V	–	BD (MILP + CP)	(400 × 8 × 40)
18	Caricato et al. (2021)	IPMSR	PS	UL	V	RD, WE	H + CP	(NA × 17 × 10)
19	Bitar et al. (2021)	UPMSR	P	UL	F	ME	MILP	(50 × 3 × 6)
20	Fang et al. (2021)	UPMR	P	UL	F	–	BD (H+MILP)	(50 × 10 × 10)
21	Al-Harkan et al. (2021)	UPMSR	P	UL	V	RD	MH	(50 × 8 × 3)
22	Kim and Lee (2021)	UniPMSR	S	UL	F	JS, ME	MILP/H	(80 × 5 × 5)
23	Yepes-Borrero et al. (2021)	UPMSR	S	UL	V	ME	H	(250 × 30)
24	Zhang et al. (2021)	UPMSR	S	UL	F	–	MH	(400 × 22 × 10)
25	Yunusoglu and Topaloglu Yildiz (2022)	UPMSR	P	UL	V	RD, ME, PC	CP	(60 × 8 × 2)
26	Chen et al. (2022)	UPMSR	P	UL	F	RD	MILP/MH	(100 × 15 × 90)
27	Soares and Carvalho (2022)	IPMSR	P	UL	F	–	MH	(200 × 10 × 17)
28	Munoz et al. (2022)	IPMSR	P	UL	F	RD	H + MILP	(100 × 10 × 24)
29	Raboudi et al. (2022)	UPMSR	S	UL	F	ME	MILP	(8 × 4 × 1)
30	Lei and He (2022)	UPMR	P	UL	V	–	MH	(350 × 30 × 150)
31	Heinz et al. (2022)	IPMSR	S	UL	F	–	CP + H	(500 × 20 × 5)
32	Avgerinos et al. (2023)	UPMSR	S	UL	F	JS	BD(H+MILP+CP)	(200 × 20 × 5)
33	Dai and Lei (2023)	UPMSR	PS	UL	V	–	MH	(250 × 20 × 100)
34	Lopez-Esteve et al. (2023)[a]	UPMSR	PS	UL	V	–	MH	(400 × 8 × 40)
35	Chen et al. (2024)	UniPMR	P	UL	V	RD	MH	(500 × 10 × –)
36	Caselli et al. (2024)[a]	IPMR	P	UL	F	RD, DD, PC, ME, WE	MILP/CP/BD	(200 × 20 × 20)
37	Li et al. (2024)	UPMR	P	UL	V	DD	MH	(500 × 8 × 10)
38	Shafiee et al. (2025)	UPMSR	P	UL	F	DD, ME, PC	MH	(40 × 7 × 41)
39	Gao et al. (2025)	UPMR	P	UL	V	PC	MILP/MH	(120 × 20 × 5)
	Our work	UPMSR	PS	L	F	RD, DD, ME	MILP/MH	(360 × 10 × 9)

BD: Bender's Decomposition DD: Due Date F: Fixed H: Heuristic I: Identical JS: Job Splitting L: Limited ME: Machine Eligibility MH: Metaheuristic PC: Precedence Constraint P: Processing PS: Processing & Setup RD: Release Date S: Setup U: Unrelated UL: Unlimited Uni: Uniform V: Variable WE: Worker Eligibility

[a] The studies to which we adapt BRKGA for benchmarking.

Table 2
Problem example data (all values are in minutes). J = Job; P = Processing time on each machine; S_M = Sequence-dependent setup times for each machine; R = Release time; D = Due time.

Job	Process		$Setup_{M1}$						$Setup_{M2}$						Release	Due
	M1	M2	1	2	3	4	5	6	1	2	3	4	5	6		
1	330	–	10	–	–	52	37	15	–	–	–	–	–	–	0	450
2	–	310	–	–	–	–	–	–	–	10	25	–	41	35	450	900
3	–	355	–	–	–	–	–	–	–	15	10	–	31	16	1350	1800
4	270	–	44	–	–	10	22	32	–	–	–	–	–	–	900	1350
5	170	300	35	–	–	45	10	44	–	36	30	–	10	60	1800	2250
6	170	145	27	–	–	12	28	10	–	37	47	–	14	10	0	2250

3.1. Problem example

We provide an example problem consisting of six jobs, two machines, and a single worker as a shared resource. The production schedule spans one week (2250 min total, 450 min per day over five work days). The resource transition limit is set to one per machine per week, allowing each machine to operate in up to two runs within the week. A *machine run* is defined as a time-continuous interval that begins when a resource (worker) sets up a machine to process a job and potentially subsequent jobs, and ends when that resource transfers to another machine. Consequently, a single machine run may include an arbitrary number of jobs, depending on the allocation and sequencing decisions. Other parameters are summarized in Table 2.

Fig. 1 illustrates both a feasible and an infeasible production schedule. In both cases, only one machine operates at a time due to the single worker, emphasizing the resource constraint. The feasible schedule respects the transition limit, whereas the infeasible one violates it on Machine 1 by including two transitions. In the UPMSR-PS with production time windows and resource transition limits, a resource transition on a machine

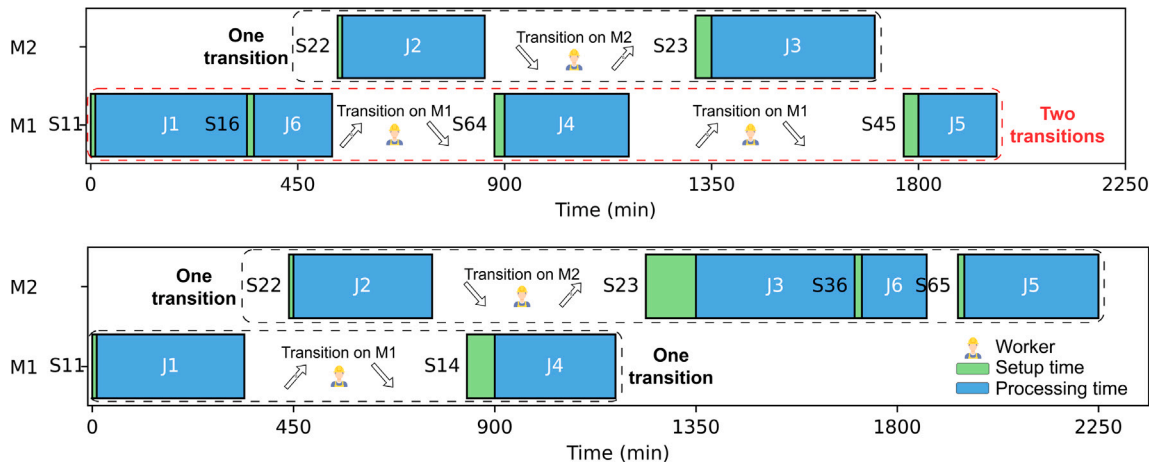


Fig. 1. Infeasible (top) and feasible (bottom) solutions for the problem example.

is defined as a pair consisting of a worker's departure from that machine followed by a subsequent arrival to the same machine, either by the same worker or by another worker. Accordingly, each departure–arrival pair constitutes one transition on that machine. Under this definition, the feasible schedule shown in the *Toy Example* satisfies the transition limit, as each machine experiences exactly one departure–arrival pair, and hence one transition. In contrast, the infeasible schedule violates the transition limit by exhibiting two such departure–arrival pairs on Machine 1, thus exceeding the allowed maximum. Both schedules also satisfy job time windows and machine eligibility constraints.

4. MILP model

Previous studies modeling PMSPs and RPMSPs with sequence-dependent setup times using MILP formulations are either time-continuous (e.g., Avgerinos et al., 2023; Bektur & Saraç, 2019; Fanjul-Peyro, 2020; Fanjul-Peyro et al., 2019) or time-indexed (e.g., Kim & Lee, 2021; Yepes-Borrero et al., 2021, 2020). In both approaches, the inclusion of setup times requires the use of Miller–Tucker–Zemlin (MTZ) constraints to eliminate subtours within the job sequence on each machine. Because our problem involves scheduling jobs while accounting for their release and due times, we adapt the MTZ formulation and use job start and end times to maintain temporal consistency among jobs. We develop both time-continuous and time-indexed formulations for the proposed model. Consistent with Floudas and Lin (2004), and Fanjul-Peyro (2020), the time-indexed model becomes unnecessarily large due to the introduction of a significant number of binary variables for each discrete time interval. We refer the reader to the appendix for the time-indexed formulation and present here the time-continuous formulation, which also accommodates floating-point (non-integer) setup and processing times.

To manage limited resource availability, Fanjul-Peyro (2020), Fanjul-Peyro et al. (2017) employ a two-dimensional bin-packing formulation to control how many machines can operate in parallel given the available resources. However, this approach is not applicable to our problem, as it does not capture the transfer of resources between machines over time. To address this limitation and explicitly manage resource transitions, we introduce the concept of discrete "buckets," defined as an input set, where each bucket represents a time-continuous interval during which a resource remains assigned to the same machine without transitioning. This idea is related to the "time slot" concept introduced by Floudas and Lin (2004), where the number of binary variables in time-indexed formulations is reduced by indexing job positions in the sequence instead of indexing discrete time instants. In our model, however, the number of buckets per machine specifies how often a machine may operate within a given time period, and each bucket can accommodate one or more jobs.

To account for the setup times between the last job of one bucket and the first job of the next, the starting and ending jobs assigned to each bucket should be explicitly tracked. We adopt the time-continuous formulation developed by Tahmassebi (1999) for the PMSP with setup times, which incorporates this structural feature by defining binary variables that identify the jobs marking the beginning and ending of the sequence on each machine. However, we modify the model by introducing both a bucket index and a time-period (week) index. Furthermore, we define two sets of decision variables: the first captures job allocation, sequencing, and timing within buckets (denoted by superscript J), and the second captures resource allocation, sequencing, and timing across buckets (denoted by superscript R). See the Appendix A for a diagram that visualizes the MILP model for the toy example of Section 3.1. The model sets, parameters, and variables are listed in Table 3, followed by the objective function and constraints.

Objective and constraints

$$\min \sum_{i\in N}\sum_{l\in M}\sum_{b\in B}\sum_{t\in T} p_{il}\, A^J_{ilbt} + \sum_{i,j\in N}\sum_{l\in M}\sum_{b\in B}\sum_{t\in T} s_{ijl}\, Q^J_{ijlbt} + \sum_{i\in N}\sum_{l\in M}\sum_{b\in B}\sum_{t\in T}\left(s_{iil}\, G^J_{ilbt} + \sum_{j\in N} s_{ijl}\, E^J_{ijlbt}\right) \tag{1}$$

$$\sum_{l\in M}\sum_{b\in B}\sum_{t\in T} A^J_{ilbt} = 1 \qquad \forall i \in N \tag{2}$$

$$\sum_{i\in N} G^J_{ilbt} \le 1 \qquad \forall l \in M,\ b \in \Delta_l,\ t \in T \tag{3}$$

$$\sum_{i\in N}\sum_{h\in N_0} E^J_{ihlbt} \le 1 \qquad \forall l \in M,\ b \in \Delta_l,\ t \in T \tag{4}$$

Table 3
An overview of sets, parameters, and decision variables of the MILP model.

Sets	
$i, j \in N = \{1, \dots, n\}$	Set of all jobs
$h \in N_0, N_0 = N \cup \{n+1\}$	Set of all jobs plus a dummy job, denoted by $n+1$
$l \in M = \{1, \dots, m\}$	Set of all machines
$k \in K = \{1, \dots, k\}$	Set of all resources
$b, u \in B = \{1, \dots, b\}$	Set of all buckets
$t \in T = \{1, \dots, t\}$	Set of time periods
$v \in T_0, T_0 = T \cup \{0\}$	Set of all time periods plus dummy period, denoted by 0
$\Gamma_l \subset N$	Subset of jobs that can be processed on machine l
$\gamma_i \subset M$	Subset of eligible machines that can process job i
$\Delta_l \subset B$	Subset of buckets belonging to machine l
$\alpha_l \subset B$	Subset of buckets referring to the first bucket of machine l
Parameters	
p_{il}	Processing time of job i on machine l, 0 if $i \notin \Gamma_l$
s_{ijl}	Setup time from job i to j on machine l; equals transfer time if $i = j$
v_t^R	Available time during time period t
m_{kbt}^R / n_{kbt}^R	Start/end of resource k's availability in bucket b, period t
$k_i / l_i \in T$	Release/due period of job i
r_{it}	Release time of job i in period t, 0 if $t \neq k_i$
d_{it}	Due time of job i in period t, v_t^R if $t < l_i$ and 0 if $t > l_i$
Variables	
$A_{ilbt \mid b \in \Delta_l}^J$	1 if job i is assigned to machine l, bucket b, period t; 0 otherwise
$Q_{ijlbt \mid i \neq j\ b \in \Delta_l}^J$	1 if job j follows i on machine l, bucket b, period t; 0 otherwise
G_{ilbt}^J	1 if job i is first on machine l, bucket b, period t
E_{ihlbt}^J	1 if job i ends bucket b and job h starts next bucket on machine l, period t
$S_{ilbt}^J / F_{ilbt}^J \geq 0$	Start/end time of job i on machine l, bucket b, period t
X_{hlv}	1 if job h is first on machine l in period $v+2$ when the machine is idle in $v+1$; 0 otherwise
A_{kbt}^R	1 if resource k is assigned to bucket b in period t; 0 otherwise
Q_{kbut}^R	1 if resource k is assigned to b before u in period t; 0 otherwise
G_{kbt}^R / E_{kbt}^R	1 if resource k starts/ends in bucket b, period t
$S_{kbt}^R / F_{kbt}^R \geq 0$	Start/end time of resource k in bucket b, period t

$$G_{ilbt}^J + \sum_{j \mid i \neq j} Q_{jilbt}^J = A_{ilbt}^J \qquad \forall i \in N, l \in M, b \in \Delta_l, t \in T \tag{5}$$

$$\sum_{h \in N_0} E_{ihlbt}^J + \sum_{j \mid i \neq j} Q_{ijlbt}^J = A_{ilbt}^J \qquad \forall i \in N, l \in M, p \in \Delta_l, t \in T \tag{6}$$

$$F_{ilbt}^J - S_{ilbt}^J \geq p_{il} A_{ilbt}^J \qquad \forall i \in N, l \in M, b \in \Delta_l, t \in T \tag{7}$$

$$F_{ilbt}^J \leq v_t^R A_{ilbt}^J \qquad \forall i \in N, l \in M, b \in \Delta_l, t \in T \tag{8}$$

$$S_{jlbt}^J - F_{ilbt}^J \geq v_t^R (Q_{ijlbt}^J - A_{ilbt}^J) + s_{ijl} Q_{ijlbt}^J \qquad \forall l \in M, b \in \Delta_l, t \in T, i \neq j \in N \tag{9}$$

$$\sum_{l \in M} \sum_{b \in \Delta_l} k_i A_{ilbt}^J \leq \sum_{l \in M} \sum_{b \in \Delta_l} A_{ilbt}^J |t| \qquad \forall i \in N, t \in T \tag{10}$$

$$\sum_{l \in M} \sum_{b \in \Delta_l} A_{ilbt}^J |t| \leq l_i \qquad \forall i \in N, t \in T \tag{11}$$

$$\sum_{l \in M} \sum_{b \in \Delta_l} r_{it} A_{ilbt}^J \leq \sum_{l \in M} \sum_{b \in \Delta_l} \sum_{t \in T} S_{ilbt}^J \qquad \forall i \in N \tag{12}$$

$$\sum_{l \in M} \sum_{b \in \Delta_l} \sum_{t \in T} F_{ilbt}^J \leq \sum_{t \in T} d_{it} \qquad \forall i \in N \tag{13}$$

The objective function (1) minimizes the total production time, accounting for job processing, transfer times, sequence-dependent setups, and final setups on each machine. Constraints (2)–(13) collectively govern the allocation, sequencing, and timing of jobs on each machine and within its bucket(s) across the time periods. Each job is allocated to exactly one bucket and one eligible machine (Constraints (2)). There is only one job at the beginning (Constraints (3)) and one at the end of the job sequence processed on every bucket of a machine (Constraints (4)). Precedence constraints (5)–(6) ensure that each job has one predecessor and one successor in the sequence. Regarding job timing, the difference between a job start and end time must be at least equal to its processing time (Constraints (7)), and the end time can only be nonzero if the job is assigned to a machine (Constraints (8)). To avoid overlapping job sequences (i.e., to eliminate sub-tours), the start time of job j must be after the end time of job i if j follows i on the same machine and bucket (Constraints (9)). Release and due constraints (10)–(13) ensure that jobs are scheduled within their specific release and due window. The model continues below.

$$\sum_{i \in N} G_{ilbt}^J = \sum_{k \in K} A_{kbt}^R \qquad \forall l \in M, b \in \Delta_l, t \in T \tag{14}$$

$$\sum_{k \in K} A_{kbt}^R \leq 1 \qquad \forall b \in B, t \in T \tag{15}$$

$$\sum_{b \in B} G_{kbt}^R \leq 1 \qquad \forall k \in K, t \in T \tag{16}$$

$$\sum_{b\in B} E^R_{kbt} \le 1 \qquad \forall k \in K, t \in T \tag{17}$$

$$G^R_{kbt} + \sum_{u|b\neq u} Q^R_{kubt} = A^R_{kbt} \qquad \forall k \in K, b \in B, t \in T \tag{18}$$

$$E^R_{kbt} + \sum_{u|b\neq u} Q^R_{kbut} = A^R_{kbt} \qquad \forall k \in K, b \in B, t \in T \tag{19}$$

$$\sum_{k\in K} F^R_{kbt} - \sum_{k\in K} S^R_{kbt} \ge \sum_{i\in N} p_{il} A^J_{ilbt} + \sum_{i,j\in N|i\neq j} s_{ijl} Q^J_{ijlbt} + \sum_{i\in N}\left(s^I_{iil} G^J_{ilbt} + s_{ihl} \sum_{h\in N} E^J_{ihlbt}\right) \qquad \forall l \in M, b \in \Delta_l, t \in T \tag{20}$$

$$F^R_{kbt} \le v^R_t A^R_{kbt} \qquad \forall k \in K, b \in B, t \in T \tag{21}$$

$$S^R_{kut} - F^R_{kbt} \ge v^R_t (Q^R_{kbut} - A^R_{kbt}) \qquad \forall k \in K, b,u \in B, t \in T, b \neq u \tag{22}$$

$$S^J_{ilbt} \ge \sum_{k\in K} S^R_{kbt} + s_{iil} G^J_{ilbt} - v^R_t (1 - A^J_{ilbt}) \qquad \forall i \in N, l \in M, b \in \Delta_l, t \in T \tag{23}$$

$$F^J_{ilbt} + s_{ihl} \sum_{h\in N_0} E^J_{ihlbt} \le \sum_{k\in K} F^R_{kbt} \qquad \forall i \in N, l \in M, b \in \Delta_l, t \in T \tag{24}$$

$$A^R_{kbt} m^R_{kbt} \le S^R_{kbt} \qquad \forall k \in K, b \in B, t \in T \tag{25}$$

$$F^R_{kbt} \le n^R_{kbt} \qquad \forall k \in K, b \in B, t \in T \tag{26}$$

Constraints (14)–(26) collectively govern the allocation, sequencing, and timing of resources on the buckets of each machine across the time periods. Constraints (14) ensures that resource k is assigned to bucket b when a job is allocated on machine l in time period t. A maximum of one resource allocated per bucket for both machine setup and job processing (Constraints (15)). Resources begin their work on one of the buckets and end it on either the same or a different bucket (Constraints (16)–(17)). Each resource is allocated on a sequence of buckets, as defined by precedence constraints (18)–(19). Regarding resource timing, the start and end times for each resource must account for the total time spent processing the jobs in their assigned bucket (Constraints (20)). A resource end time on a bucket can only have a non-zero value if allocated to a machine (Constraints (21)). To prevent overlapping resource allocation (i.e., to eliminate sub-tours), constraints (22) ensure a resource starts work on bucket u only after completing its work on bucket b, if buckets b and u are assigned consecutively to the same worker k. Each job start time must follow the start time of the resource responsible for processing that bucket, with consideration for transfer times (Constraints (23)). Each job completion time must occur before the finish time of the resource processing that bucket, reduced by the setup time required to process the first job in the next bucket of the same machine (Constraints (24)). The start and finish times of each resource must fall within their available time windows (Constraints (25)–(26)). The model continues below.

$$\sum_{k\in K} A^R_{kb+1t} \le \sum_{k\in K} A^R_{kbt} \qquad \forall b, b+1 \in \Delta_l, l \in M, t \in T \tag{27}$$

$$\sum_{k\in K} S^R_{kb+1t} \ge \sum_{k\in K} F^R_{kbt} + v^R_t \left(\sum_{k\in K} A^R_{kb+1t} - \sum_{k\in K} A^R_{kbt}\right) \qquad \forall k \in K, b \in \Delta_l, t \in T \tag{28}$$

$$G^J_{hlb+1t} \ge \sum_{i\in N} E^J_{ihlbt} + \left(\sum_{k\in K} A^R_{kb+1t} - \sum_{k\in K} A^R_{kbt}\right) \qquad \forall h \in N_0, l \in M, b, b+1 \in \Delta_l, t \in T \tag{29}$$

$$\sum_{i\in N}\sum_{b\in\Delta_l} E^J_{ihlbt} - \sum_{b,b+1\in\Delta_l} G^J_{hlb+1t} + X_{hlt-1} = G^J_{hl\alpha_l t+1} + X_{hlt} \qquad \forall h \in N_0, l \in M, t < |T| \tag{30}$$

Constraints (27)–(30) collectively govern the precedence, timing, and connectivity of a machine buckets both within and across time periods. Buckets on a machine within a time period must be utilized in ascending order, ensuring bucket $b+1$ is not used unless bucket b on the same machine is already in use (Constraints (27)) and the start time of bucket $b+1$ succeeds the end of bucket b (Constraints (28)). The connectivity between job sequences of active buckets on a machine within a time period is maintained by constraints (29). Constraints (30) ensures no closed loops (i.e, subtours) are formed in the job sequence of a bucket and manages connectivity across time periods by activating an auxiliary variable when no job is scheduled in time period t, shifting the next job h from period $t-1$ to the period $t+1$.

4.1. Two-step solution approach for the MILP model

As shown in Algorithm 1, we begin by solving a relaxed version of the MILP model, where the job allocation constraints (2) are relaxed from an equality to an inequality, and the objective is changed from minimizing total production time to maximizing the number of scheduled jobs. This step continues until the optimality gap reaches zero or the time limit is reached. If no feasible solution is found, the second step is skipped, and the MILP model is deemed unsuccessful in solving the instance. If the solver reaches a zero gap, one of two outcomes occurs: (1) all jobs are allocated, yielding a feasible solution that serves as a warm start for the second step; or (2) only a subset is allocated, confirming infeasibility, and no further steps are taken. In the second step, if applicable, the warm-start solution is used to solve the model described in Section 4. The performance of the proposed two-step approach is compared against the single-step model in Appendix A.

5. BRKGA

The BRKGA (Gonçalves & Resende, 2011) is a variant of the traditional genetic algorithm (GA), see e.g. Mitchell (1998), designed to address the limitations of conventional GAs. Unlike standard GAs that typically use binary, integer, or domain-specific representations, the BRKGA encodes solutions as vectors of floating-point numbers in the range $[0, 1]$, known as *random keys*. These keys are interpreted by a problem-specific decoder that constructs a feasible solution and evaluates its fitness. This separation of solution encoding and problem logic allows BRKGA to apply standard

Algorithm 1 Two-step MILP solution approach

Inputs: τ_1, τ_2

Step 1 (job-allocation maximization):
Relax the job-allocation constraints of the MILP model: $\sum_{l\in M}\sum_{b\in B}\sum_{t\in T} A^J_{ilbt} \leq 1 \quad \forall i \in N$
Replace the objective function with: $Z = \max \sum_{i\in N}\sum_{l\in M}\sum_{b\in B}\sum_{t\in T} A^J_{ilbt}$
Solve the relaxed MILP for at most τ_1 Seconds.
if $Z^* < |N|$: **then**
    return failure: MILP could not find a feasible solution in the given time τ_1.
else if $Z^* = |N|$: **then**
    Step 2 (time minimization):
    Solve the MILP model of Section 4 using the Step-1 solution as a warm start with at most $\tau_2 - \tau_1$ Seconds.
    return feasible: the schedule and objective value

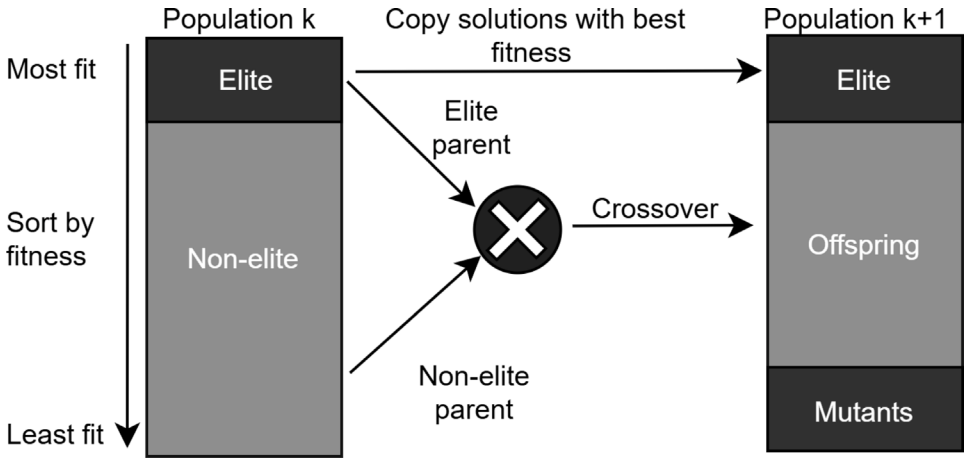


Fig. 2. The population of generation $k + 1$ in BRKGA is created by retaining elite solutions, adding randomly generated mutants, and performing crossover between elite and non-elite solutions.
Source: Figure adapted from Gonçalves and Resende (2011).

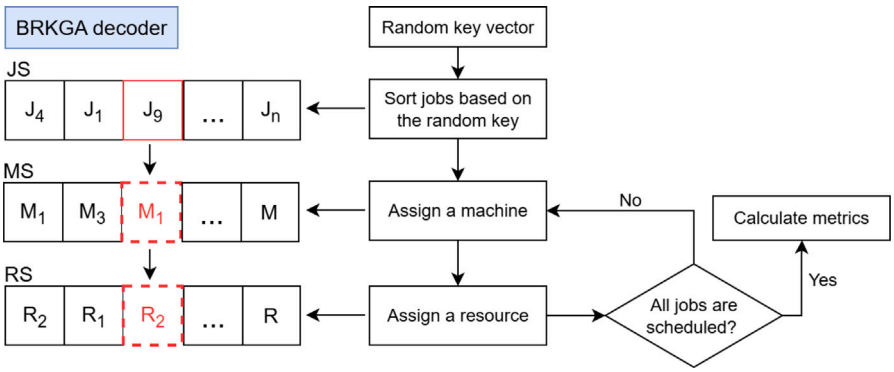


Fig. 3. The BRKGA decoder workflow.

genetic operators, such as uniform crossover and mutation, without the need for customizations to maintain feasibility, which is often necessary in traditional GAs, especially for variable-length solutions.

The BRKGA is particularly well-suited for solving the UPMSR-PS due to its ability to effectively handle complex constraints, including resource availability, transition limits, and job production windows. In such settings, construction heuristics, local search and neighborhood-based methods are prone to generating infeasible solutions. The BRKGA overcomes this by embedding feasibility considerations within the decoder and/or by penalizing infeasible outcomes (Hottung & Tierney, 2016).

A schematic overview of the evolutionary process is shown in Fig. 2. At each generation, the BRKGA population is divided into *elite* and *non-elite* individuals. The elite set is comprising the top-performing solutions and preserved unchanged in the next generation to retain high-quality solutions. New individuals are generated in two ways: (1) offspring are produced via a biased uniform crossover (Gonçalves & Resende, 2011), where each element is inherited from the elite parent with a probability greater than 0.5, ensuring a stronger influence from superior solutions; and (2) the remaining population is filled with *mutants*, i.e., randomly generated key vectors introduced to maintain genetic diversity and reduce the risk of premature convergence. These components together enable BRKGA to effectively balance exploration and exploitation. We use the BRKGA without modifications to the core algorithm described by Gonçalves and Resende (2011) and the readers are referred to the this study to find the BRKGA pseudocode.

Table 4
An overview of parameters and variables used in the BRKGA decoder.

Parameters	
os_k^R/oe_k^R	Start/end of off time for resource k
r_j/d_j	Release/due time of job j
α	Weight of objectives
β	Weekly available time
δ	Weekly transition limit per machine
Variables	
JS	Job sequence list
CR_k	Completion time of resource k in the current decoding step
CCR_k	Temporary copy of CR_k
C_l	Completion time of machine l in the current decoding step
M^*/R^*	Assigned machine/resource to the current job
IS	Transfer time
P_l/S_l	Processing/sequence-dependent setup time of the current job on machine l
T_l	Sum of processing and setup time of the current job on machine l
RM	Remaining eligible machine set for the current job
$WT_{l,t}$	Transitions on machine l during week t
TN	Total excess transitions beyond limit δ

5.1. BRKGA decoder

A novel BRKGA decoder is developed to solve the UPMSR-PS with the constraints defined in Section 3. The same sets defined in Table 3 are used. The parameters, and variables specific to the decoder are summarized in Table 4. The overall workflow of the decoder is illustrated in Fig. 3, and the corresponding pseudocode is presented in Algorithm 2. The decoder maps a random key vector x to a schedule and returns its associated fitness value. The vector x has length equal to the number of jobs, where each element is a floating-point number in the interval $[0, 1]$ representing the scheduling priority of the corresponding job. The fitness value is computed as a weighted sum of the total production time, penalized total tardiness, and penalized total number of resource transitions exceeding the weekly limit δ. Although infeasible solutions are temporarily allowed to help the algorithm escape local optima and explore the broader solution space, the final solution must achieve zero tardiness and no excessive resource transitions to be considered feasible. Other constraints, such as resource availability limiting parallel machine operations, job release times, and machine eligibility, are inherently respected by the decoder, eliminating the need for explicit penalty terms in the fitness evaluation. Alternative decoding approaches are discussed in the Appendix A. Here, we present the best-performing decoder.

As shown in Algorithm 2, the decoder starts by sorting the job list JS in ascending order of the random keys and initializing the completion times of all machines and resources to zero. The main loop (Lines 8–37) processes the jobs sequentially according to this order and assigns each job j to a machine M^* and a resource R^*. For each job j, the decoder first evaluates all eligible machines $l \in \gamma_j$ (Lines 9–15). For each machine, the production time T_l and the completion time C'_l of job j are computed by considering the setup time, processing time, current machine completion time, and the job release time. The set of eligible machines is then copied into a dynamic candidate list RM (Line 15).

A greedy search is subsequently performed over RM (Lines 16–26) to identify a feasible machine–resource pair. In each iteration, the machine with the minimum production time T_l, using completion time C'_l as a tie-breaker, is selected (Line 17). Given this machine, the ResourceAssign heuristic (Line 18) selects a candidate resource, as described in Section 5.1.1. The feasibility of this assignment is then evaluated with respect to the weekly resource transition limit (Lines 19–26). Specifically, the current time period t is identified (Line 19), and if the selected resource is assigned to a different machine than its previous assignment (Line 20), the corresponding weekly transition counter $WT_{M^*,t}$ is incremented (Line 21). The assignment is accepted only if the transition limit δ is not exceeded (Line 22); otherwise, the machine is removed from RM (Line 23) and the search continues.

If no feasible machine–resource pair is found for job j (i.e., $RM = \emptyset$), a fallback strategy is applied (Lines 27–32). In this case, the machine with the minimum production time and completion time is selected, a resource is assigned using ResourceAssign, and both the weekly transition counter and the total number of excessive transitions TN are incremented since no machine–resource pair without violating the transition limit δ was identified from the previous step.

Finally, when a transition occurs, the transfer time is added to the current setup time (S_{M^*}) (Line 33). The completion times of the selected machine and resource are then updated (Lines 34–35),followed by the accumulation of the total production time and total tardiness (Lines 36–37). After all jobs have been scheduled, the decoder returns the fitness value of the constructed schedule (Line 38).

5.1.1. Resource assignment heuristic

The ResourceAssign heuristic, presented in Algorithm 3, selects an appropriate resource R^* for a job that has already been assigned to a candidate machine M^*. The heuristic first checks whether a resource was previously assigned to M^* (Line 2). If such a resource exists, it is reused to avoid unnecessary transitions (Line 3). Otherwise, the resource whose completion time CR_k is closest to the completion time of machine M^* is selected (Line 4).

Once the resource R^* is chosen, its projected completion time after executing job j is computed by accounting for the machine completion time, setup time, processing time, and job release time (Line 5). If this projected execution overlaps with a resource unavailability window, defined by its start $OS^R_{R^*}$ and end $OE^R_{R^*}$, the resource availability CR_{R^*} is shifted to the end of the unavailability period to prevent infeasible assignments (Lines 6–7). The heuristic then returns the selected resource and its completion time.

Algorithm 2 Decoder heuristic

function DECODER(x) ▷ Decode a random key vector x into a schedule
  Initialize:
  $JS \leftarrow JobList[1, \ldots, N]$
  $JS \leftarrow \text{argsort}(x)$ ▷ Sort jobs in nondecreasing order of their random keys in x
  $C_l \leftarrow 0$ for all machines l
  $CR_k \leftarrow 0$ for all resources k
  $ProductionTime \leftarrow 0,\ Tardiness \leftarrow 0,\ TN \leftarrow 0$
  for each job $j \in JS$ **do**
    for each machine $l \in \gamma_j$ **do**
      Compute the sequence-dependent setup time S_l and processing time P_l on machine l for job j.
      $T_l \leftarrow S_l + P_l$
      if $C_l + S_l \leq r_j$ **then** ▷ Compute completion time of job j on machine l
        $C'_l \leftarrow r_j + P_l$
      else $C'_l \leftarrow C_l + S_l + P_l$
    $RM \leftarrow \gamma_j$
    while $RM \neq \emptyset$ **do** ▷ Loop while there are remaining eligible machines
      $M^* \leftarrow \arg\min_{l \in RM} (T_l,\ C'_l)$ ▷ Select machine with minimum T_l; use C'_l as tie-breaker
      $R^*,\ CR'_{R^*} \leftarrow$ **ResourceAssign**$(CR,\ C_{M^*},\ S_{M^*},\ P_{M^*})$
      Find current time period (week): $t \leftarrow \left\lfloor \frac{\max(C_{M^*},\ CR'_{R^*})}{\beta} \right\rfloor$
      if resource R^* is assigned to a different machine than its last assignment **then**
        $WT_{M^*,t} \leftarrow WT_{M^*,t} + 1$
        if $WT_{M^*,t} \geq \delta$ **then** ▷ Check if transition limit exceeded
          Remove M^* from the remaining machine set: $RM \leftarrow RM \setminus \{M^*\}$
          Undo the increment: $WT_{M^*,t} \leftarrow WT_{M^*,t} - 1$
        else break ▷ Accept M^* and stop evaluating other machines
      else break ▷ No transition; accept M^*
    if $RM = \emptyset$ **then** ▷ Fallback if no machine without transition is feasible
      $M^* \leftarrow \arg\min_{l \in \gamma_j} (T_l,\ C'_l)$
      $R^*,\ CR'_{R^*} \leftarrow$ **ResourceAssign**$(CR,\ C_{M^*},\ S_{M^*},\ P_{M^*})$
      Find current time period (week): $t \leftarrow \left\lfloor \frac{\max(C_{M^*},\ CR'_{R^*})}{\beta} \right\rfloor$
      $WT_{M^*,t} \leftarrow WT_{M^*,t} + 1$
      $TN \leftarrow TN + 1$ ▷ Increase TN since a transition is required
    if transition occurred **then** $S_{M^*} \leftarrow S_{M^*} + IS$ ▷ Add the transfer time IS to S_{M^*}
    $C_{M^*} \leftarrow \max(\max(CR'_{R^*},\ C_{M^*}) + S_{M^*},\ r_j) + P_{M^*}$
    $CR_{R^*} \leftarrow C_{M^*}$
    $ProductionTime \leftarrow ProductionTime + T_{M^*}$
    $Tardiness \leftarrow Tardiness + \max(0,\ C_{M^*} - d_j)$
  return fitness value: $\alpha_1 \cdot ProductionTime + \alpha_2 \cdot Tardiness + \alpha_3 \cdot TN$

Algorithm 3 Resource assignment heuristic

function RESOURCEASSIGN($CR, C_{M^*}, S_{M^*}, P_{M^*}$)
  if the last resource R assigned to machine M^* exists **then**
    $R^* \leftarrow R$ ▷ Reuse the last resource previously assigned to M^*
  else $R^* \leftarrow \arg\min_{k \in K} |CR_k - C_{M^*}|$ ▷ Select the resource whose availability is closest to C_{M^*}
  $CCR_{R^*} \leftarrow \max\left(\max\left(CR_{R^*}, C_{M^*}\right) + S_{M^*}, r_j\right) + P_{M^*}$ ▷ Project completion time of R^*
  if $CCR_{R^*} \geq OS^R_{R^*}$ **and** $CR_{R^*} \leq OE^R_{R^*}$ **then**
    $CR_{R^*} \leftarrow OE_{R^*}$ ▷ Shift to the end of resource unavailability window
  return R^*, CR_{R^*} ▷ Return the selected resource and its updated completion time

6. Computational experiments

The MILP model is executed on AIMMS 24.5.5.4 software using the Gurobi 12.0.1 solver. The BRKGA and its decoder are both written in Python. All experiments conducted in this study, including those in the comparative analysis sections, were performed on a single machine with the following specifications: Windows 10 64-bit operating system, an AMD Ryzen 9 3900X processor (12 cores, 24 threads) @ 4.6 GHz processor, and 64 GB of RAM. We offer all codes in an online repository.[1]

[1] https://github.com/Mazzy96/BRKGA4RPMSPs

Table 5
Descriptions and tuned values of the BRKGA parameters.

Description	Tuned value
Size of the population	391
Percentage of elite population	18.67%
Percentage of mutant population	10.40%
Bias favoring the elite parent during crossover	0.61

6.1. Random instance generation

Based on a real-world food processing case study, synthetic instances are generated using combinations of input parameters, including the number of jobs $J \in \{30, 60, 90, 120, 180, 360\}$, number of machines $M \in \{2, 4, 8, 10\}$, scheduling horizon $W \in \{1, 3, 6\}$ weeks, resource capacity $R \in \{1, 3, 6, 9\}$, proportion of jobs with production time windows $TW \in \{0, 0.5, 1\}$, and machine eligibility ratios $ME \in \{0.6, 0.8, 1\}$. Following guidelines from the work of Fanjul-Peyro et al. (2019), instances ensure a minimum job-to-machine-week ratio of 15, reflecting high-utilization production settings. Personnel resources are always fewer than machines, with the machine-to-resource ratio capped at three. When time windows are included, each job receives distinct release and due times. Similarly, when machine eligibility constraints apply, jobs are evenly distributed across the eligible machines. The maximum allowable resource transition per week per machine is set to one in all instances. Details on the generation of setup times, processing times, and weekly resource availability are provided in the Supplementary Materials.

6.2. BRKGA configuration

The BRKGA parameters, listed in Table 5, include both continuous and integer values. Given the high computational cost of examining a full factorial design, we use the open-source OPTANO Algorithm Tuner (OAT)[2] to efficiently configure the BRKGA parameters. OAT is based on the gender-based genetic algorithm (GGA) (Ansótegui et al., 2009) and is developed by Kemminer et al. (2024). We tune the parameters using 80% of the synthetic instances and validate them on the remaining 20%. The goal is set to maximize the final fitness value of the BRKGA over a fixed runtime. BRKGA is run for $2 \times n$ seconds per instance, where n is the number of jobs, as preliminary analysis shows convergence typically occurs within this time threshold. The tuned parameter values are presented in Table 5.

6.3. Experimental results

Table 6 presents the results of MILP and BRKGA across 243 synthetic instances. Each row aggregates nine instances with identical numbers of jobs, machines, resources, and weeks, covering all combinations of machine eligibility and job production time window ratios. The MILP solver is given a 62-minute time limit per instance: 60 min for Steps I and II together, plus an additional 2 min to compute the Step II lower bound if Step I uses the full 60 min. The BRKGA is allocated a time budget of $2 \times n$ seconds per population for all instances, where n is the number of jobs. Following Gonçalves and Resende (2011), the BRKGA uses 20 independent populations in parallel, each on a separate CPU thread with a unique seed. If no feasible solution is found, additional batches of 20 populations are launched incrementally, up to a total time limit of 30 min. The reported BRKGA solution time reflects the cumulative time taken to run batches until a feasible solution was identified. Instances for which no feasible solution is found by the BRKGA are excluded from the average BRKGA solution time.

For the MILP, the reported solution time and optimality gap are computed only over instances for which a feasible solution is found within the time limit. Specifically, the MILP solution time is the cumulative computation time of Steps I and II defined in Section 4.1, corresponding to the time required to obtain a feasible solution and either prove optimality or reach the time limit. The MILP optimality gap is defined as the difference between the best feasible solution and the best lower bound returned by the solver; instances without feasible solutions are omitted from the gap calculation. The MILP finds feasible solutions for 59.8% of the instances and proves optimality (zero gap) for 33.2% of the instances, with an average optimality gap of 1.28% across all feasible instances. Its feasibility performance declines with increasing problem size, particularly beyond 120 jobs.

We evaluate BRKGA solution quality using the relative percent deviation (RPD), defined as $100 \times \frac{z_{\text{BRKGA}} - z_{\text{MILP}}}{z_{\text{MILP}}}$. RPD is computed only for instances where both the BRKGA and MILP return feasible solutions, using the best objective value found by the MILP. Instances with no feasible solution are excluded. For each instance class, the reported RPD is the average over all applicable instances, and overall averages are reported as weighted means with weights equal to the number of feasible instances solved by the corresponding method in each class.

The BRKGA finds feasible solutions for 89.8% of the instances, compared with 59.8% obtained by the MILP, and achieves an average RPD of 1.71% (maximum 7.29%). As the problem size increases, the MILP yields substantially fewer feasible solutions, demonstrating the scalability of the BRKGA. For the 180 job instances, the MILP finds feasible solutions for 64% of the instances, whereas the BRKGA solves 94% of the instances. This difference becomes more pronounced for the 360 job instances, where the MILP obtains feasible solutions for only 17% of the instances compared with 79% for the BRKGA. Overall, the BRKGA returns more feasible solutions, specially for large instances with release and due-date constraints where the MILP often fails to find a feasible solution.

Across the instance sizes with up to 120 jobs, 180 jobs, and 360 jobs, the average RPD values remain below 2% (1.67%, 2.03%, and 0.72%, respectively), showing that the BRKGA maintains solution quality close to that of the MILP. For some large instances (180 and 360 jobs), the RPD becomes negative, indicating that the BRKGA can outperform the MILP on solution quality, considering the one-hour solution time limit. In terms of solution time, the MILP requires about 32 min on average per feasible instance to find a feasible solution and then optimize it or certify optimality. The BRKGA has a shorter solution time of 7 min on average and often requires only a single batch of 20 parallel populations to yield a feasible solution. Overall, the BRKGA offers better scalability and shorter solution times while maintaining solution quality close to the MILP.

[2] OAT is available at https://github.com/OPTANO/optano.algorithm.tuner (accessed on April 1, 2026)

Table 6
Performance comparison of MILP (Gurobi) model and BRKGA algorithm.

Instances		MILP				BRKGA		
(J, M, R, W)	#	# Feasible (Infeasible)	# Optimal solutions	Avg. solution time (min)	Avg. optimality gap (%)	# Feasible	Avg. solution time (min)	Avg. RPD (%)
(30,2,1,1)	27	27 (0)	26	0.05	0.00	27	1.00	0.48
(60,2,1,1)	27	22 (5)	22	4.22	0.00	22	2.00	1.36
(60,4,3,1)	27	27 (0)	18	18.11	0.08	27	2.00	0.70
(90,2,1,1)	27	23 (4)	23	3.78	0.00	23	3.91	0.80
(90,2,1,3)	27	27 (0)	14	34.75	0.21	27	3.00	1.77
(90,4,3,1)	27	26 (0)	16	27.83	0.27	27	3.00	1.32
(120,2,1,1)	27	21 (6)	20	8.06	0.00	18	9.17	0.93
(120,2,1,3)	27	25 (1)	8	48.73	0.34	26	5.23	2.85
(120,4,3,1)	27	24 (0)	9	39.25	0.44	27	4.00	1.80
(120,8,3,1)	27	19 (0)	9	33.85	0.51	27	4.00	5.59
(120,8,6,1)	27	20 (0)	11	31.47	0.21	27	4.00	1.61
(180,2,1,3)	27	20 (0)	4	55.43	0.47	19	6.00	2.31
(180,2,1,6)	27	22 (0)	0	62.06	2.54	25	7.08	2.03
(180,4,3,1)	27	17 (0)	10	30.52	0.85	27	6.00	1.80
(180,4,3,3)	27	17 (0)	0	62.13	5.83	27	6.00	−3.25
(180,8,3,1)	27	15 (0)	9	30.27	0.99	25	7.80	7.29
(180,8,6,1)	27	17 (0)	9	33.92	0.58	27	6.00	2.53
(180,10,6,1)	27	19 (0)	10	34.86	0.19	27	6.00	3.13
(180,10,9,1)	27	11 (0)	8	17.40	0.16	27	6.00	0.25
(360,4,3,3)	27	1 (0)	0	62.21	13.47	21	13.00	−10.31
(360,4,3,6)	27	0 (0)	0	–	–	18	12.00	–
(360,8,3,1)	27	7 (0)	1	59.12	14.53	16	13.50	2.83
(360,8,3,3)	27	0 (0)	0	–	–	15	12.67	–
(360,8,6,1)	27	8 (0)	4	49.52	0.13	27	12.00	4.36
(360,8,6,3)	27	0 (0)	0	–	–	22	14.18	–
(360,10,6,1)	27	12 (0)	5	51.95	11.67	27	12.00	0.50
(360,10,9,1)	27	9 (0)	6	45.99	4.83	27	12.00	−2.64
Total/Avg.	**729**	**436 (16)**	**242**	**31.60**	**1.28**	**655**	**6.84**	**1.71**

Table 7
Average RPD of BRKGA compared to GRASP on UPMSR-PS instances.

Small instances				Medium instances				Large instances			
(J, M)	RPD (%)	(J, M)	RPD (%)	(J, M)	RPD (%)	(J, M)	RPD (%)	(J, M)	RPD (%)	(J, M)	RPD (%)
(10, 4)	0.00	(30, 8)	−0.26	(60, 4)	−0.03	(100, 8)	0.81	(200, 4)	−0.03	(300, 8)	−0.03
(10, 6)	0.00	(40, 4)	−0.99	(60, 6)	−0.02	(140, 4)	−0.05	(200, 6)	−0.36	(350, 4)	−0.14
(10, 8)	0.00	(40, 6)	−0.32	(60, 8)	−0.11	(140, 6)	0.22	(200, 8)	0.25	(350, 6)	−0.44
(20, 4)	0.96	(40, 8)	1.00	(80, 4)	−0.59	(140, 8)	0.17	(250, 4)	−0.03	(350, 8)	−0.60
(20, 6)	−0.40	(50, 4)	−0.04	(80, 6)	−0.24	(120, 4)	0.03	(250, 6)	−0.08	(400, 4)	−0.32
(20, 8)	0.00	(50, 6)	0.06	(80, 8)	0.13	(120, 6)	0.46	(250, 8)	0.03	(400, 6)	0.03
(30, 4)	−0.32	(50, 8)	−0.37	(100, 4)	−0.21	(120, 8)	0.18	(300, 4)	−0.34	(400, 8)	−0.10
(30, 6)	1.14			(100, 6)	−0.18			(300, 6)	−0.17		
Avg. RPD		0.03%		0.04%				−0.15%			
Overall avg. RPD:						**−0.03%**					

6.4. Comparative analysis of BRKGA and GRASP on the UPMSR-PS

We benchmark our BRKGA against the state-of-the-art heuristic framework, based on the GRASP algorithm of Lopez-Esteve et al. (2023), one of the leading metaheuristics for the UPMSR-PS. While their problem formulation is similar, it omits job production windows, machine eligibility, and resource transition constraints, resulting in a less restrictive solution space. For details on their problem, the GRASP algorithm, and instance generation, we refer to the work of Lopez-Esteve et al. (2023).

We adapt the framework of Lopez-Esteve et al. (2023) by replacing their GRASP metaheuristic with our BRKGA. Their greedy constructive heuristic and local search are retained to generate an initial solution, which warm starts the BRKGA by embedding job start and end times as random keys in the initial population. After the BRKGA execution, the best solution is repaired using the same repair algorithm as the one developed by Lopez-Esteve et al. (2023). Details of the BRKGA decoder, tuned parameters, and an ablation study on the warm-start approach are provided in the Supplementary Materials. We use the GRASP parameter settings from the original study and tune the BRKGA parameters using OAT. Both methods are allocated $3 \times n$ seconds of CPU time, where n is the number of jobs.

To compare the performance of BRKGA and GRASP, we calculate the RPD as follows: $100 \times \frac{z_{\text{BRKGA}} - z_{\text{GRASP}}}{z_{\text{GRASP}}}$. Since all instances in this benchmark are feasible for both methods, the reported RPD values in Table 7 are averaged over all instances within each class. According to Table 7, the BRKGA matches GRASP on the UPMSR-PS benchmark, showing a slight average makespan improvement of −0.15% on large instances and −0.03% across all instances. The results in Lopez-Esteve et al. (2023) indicated the GRASP achieves solutions within 1% of the best known solutions reported by Fanjul-Peyro (2020). Similarly, our results indicate that the BRKGA consistently yields near-optimal solutions for the UPMSR-PS.

The RPDs in Table 7 provide a descriptive statistical analysis of the two algorithms, confirming the similar performance of both on the UPMSR-PS. Since RPD values are not normally distributed and the homoscedasticity assumption required for parametric analysis of variance (ANOVA) is not satisfied, we apply a nonparametric test. We use the Wilcoxon signed-rank test (Conover, 1999) to evaluate whether the median of the RPD

Table 8
Summary of statistical tests on the RPD values of BRKGA Vs. GRASP.

Test	Factor	Statistic	*p*-value
Wilcoxon signed-rank	Algorithm (BRKGA vs. GRASP)	Median RPD = −0.00024	0.1144
Rank-based ANOVA	Problem size (small/med/large)	$F = 1.319$	0.2696
Rank-based ANOVA	Number of machines	$F = 1.428$	0.2421
Rank-based ANOVA	Size × Machines interaction	$F = 0.738$	0.5669

between BRKGA and GRASP differs from zero. This corresponds to testing the null hypothesis that the two algorithms perform equivalently on paired instances. The test reveals no statistically significant difference. To assess whether instance characteristics influence the relative performance of the algorithms, a rank-based two-way ANOVA is performed following the Conover–Iman procedure (Conover, 1999). Neither problem size, number of machines, nor their interaction has a statistically significant effect on RPD. The numerical results of all hypothesis tests are summarized in Table 8.

The BRKGA not only statistically matches the GRASP approach in solution quality, but is also easier to extend to more complex problems, as demonstrated by its adaptation to our problem and to the IPMR-P with complex side constraints presented in Caselli et al. (2024) (see Section 6.5). In our problem, namely the UPMSR-PS with job time window and resource transition limit constraints, both sets of constraints must be satisfied simultaneously, and fixing a violation of one constraint may lead to violating the other. The BRKGA handles these constraints jointly within its decoder by effectively exploring the solution space and guiding solutions toward feasible regions. As a result, it consistently generates high-quality feasible solutions under complex real-world constraints. While GRASP is conceptually simpler, as adapted in the work of Lopez-Esteve et al. (2023) for the UPMSR-PS without job time windows and resource transition limits, adapting GRASP to more complex problems, including ours, is non-trivial. The construction and repair heuristics in GRASP struggle with tightly coupled constraints, since resolving one constraint violation often induces another. Finally, metaheuristics should be easy to modify when problem characteristics change, signifying the superiority of the BRKGA versus GRASP. GRASP has limited ability to accommodate temporal constraints such as job release and due times, either when such constraints are added to the UPMSR-PS presented by Lopez-Esteve et al. (2023) or when they are present in our problem. In contrast, the BRKGA, as demonstrated in the experimental results, can effectively solve instances both with and without these constraints.

6.5. Comparative analysis of BRKGA with exact methods on the IPMR-P with complex side constraints

Demonstrating BRKGA capability in solving RPMSPs with side complex constraints, we apply the BRKGA to the identical PMR-P (IPMR-P) with workforce, job release/due time, and precedence/contiguity constraints studied by Caselli et al. (2024). We execute both their Bender's decomposition and MILP models, as the authors noted that neither consistently outperforms the other. Due to requiring high memory volume by both exact models, we run them sequentially to avoid resource contention. Following authors guideline, each exact model is initialized using CP-derived solutions. Readers are referred to work of Caselli et al. (2024) to find more on the details on the problem definition, computational setting, and random instance dataset, which we similarly adopt in the benchmarking.

The BRKGA is adapted for this problem using warm-start initialization and executed with 20 independent populations in parallel across available CPU threads. Each population was run for $2 \times n$ seconds, where n is the number of jobs, followed by a local search phase to refine the best solutions. The entire process, including local search, was limited to 30 min to match the CPU time limit used in the work of Caselli et al. (2024). Detailed descriptions of the warm-start procedure, BRKGA decoder, local search, and tuned BRKGA parameters are provided in the Supplementary Materials.

Benchmarking was performed using publicly available random instances from the work of Caselli et al. (2024), with results summarized in Table 9. Each row represents a set of 10 instances with identical job, machine, and worker counts but varying percentage of eligibility constraints and ratio of precedence/contiguity relations. Since some optimal solutions in this benchmark have zero total tardiness, the RPD is not applicable. Therefore, Table 9 reports the average difference in total weighted tardiness between the BRKGA and the exact methods, along with the corresponding solution times.

Overall, BRKGA achieves comparable performance to the exact methods, often with slightly higher weighted tardiness, except in one highly constrained large-scale case (200 jobs, 8 machines, 8 resources), where it significantly outperforms exact methods both in solution quality and runtime. The BRKGA solution time, set to twice the number of jobs (in seconds), remains relatively stable for instances of the same size, with any remaining time used for local search. In contrast, the solution time of the exact algorithms tends to increase as the number of available machines and workers decreases, partly due to the additional time required to optimize a feasible solution or prove its optimality. These results highlight the effectiveness of the BRKGA, particularly for large or highly constrained instances where exact methods face scalability limitations and long solution times.

6.6. Case study

The case study focuses on a food processing plant producing over 100 powdered products across three production lines, each dedicated to specific product types, introducing machine eligibility constraints. Each line includes five sequential stages, with packing being the bottleneck due to longer processing and setup times. Thus, scheduling focuses on the packing stage, modeled as a parallel machine scheduling problem. Due to personnel limits, only two of the three packing machines can run simultaneously, introducing "additional resource" constraints. Furthermore, plant policy restricts each line to one weekly run to reduce availability losses, translating into a zero transition limit per machine per week. The planning horizon spans three weeks, with 90 jobs scheduled across three packing machines. Jobs vary in batch size (1–16), and their processing time includes batch packing, to same product setup, and initial machine setup. All jobs have a release time of zero due to sufficient supply of raw materials, while due times depend on customer deadlines. Machine 1 and 2 process the bulk of jobs (34 and 48), while Machine 3 handles four jobs. An overview of input parameters for the case study is provided in Table 10.

Setup times are identified from overall equipment effectiveness (OEE) reports as major downtime sources, roughly a third of availability losses. Improvements in job sequencing, such as campaign manufacturing (grouping same-product jobs), can reduce these inefficiencies, saving time, cost, and energy. Due to a short solution time, the MILP model is implemented without a warm-start and solved the case optimally in under two

Table 9
Performance comparison of exact and BRKGA methods.

Instances		Exact		BRKGA		Difference in
(J, M, W)	#	Avg. weighted tardiness	Avg. solution time (min)	Avg. weighted tardiness	Avg. solution time (min)	avg. weighted tardiness
(50, 2, 2)	10	1247	19.2	1280	1.7	33
(50, 5, 3)	10	12	50.7	18	0.8	7
(50, 5, 5)	10	1	0.1	2	0.4	1
(100, 4, 4)	10	582	39.3	857	3.7	275
(100, 10, 5)	10	0	0.0	5	2.4	5
(100, 10, 10)	10	0	0.0	0	0.7	0
(200, 8, 8)	10	3324	48.0	1598	14.5	−1726
(200, 20, 10)	10	0	0.4	10	8.2	10
(200, 20, 20)	10	0	0.1	0	4.9	0
Total/Avg.	**90**	**574**	**17.5**	**419**	**4.2**	**−155**

Table 10
An overview of input parameters for the case study.

Number of jobs	90	Weekly time availability	5 days × 450 min/day = 2250 min
Number of machines	3	Job batch size	[1, 16]
Number of personnel	2	Job processing time	[5, 263] min
Planning horizon	3 weeks	Setup time (to different product)	[25, 60] min
Resource transition limit	0	Setup time (to same product)	7.2 min

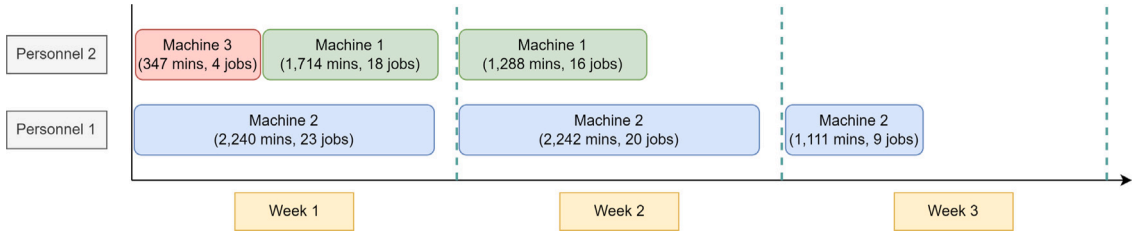


Fig. 4. Production schedule for the case study.

minutes, resulting a total production time of 149 h. The BRKGA achieves a solution within 4.5 min, with an RPD of 1.56% and 2.3 additional setup hours compared to MILP solution. For subsequent analyses, we rely on the MILP model to ensure solution optimality and accuracy across scenario analysis.

The Gantt chart in Fig. 4 shows the production schedule over the three-week horizon. Personnel and machines are fully utilized in the first week but idle times are observed for both in weeks two and three. This unused capacity can be utilized to: (1) accept additional orders, (2) perform deep cleaning (requiring two days per machine), or (3) grant personnel leave to accommodate their potential time-off requests.

6.6.1. *Analysis of personnel shift planning*

While the main purpose of the MILP and BRKGA methods is to generate optimal production schedules, they also offer valuable insights for personnel shift planning. In our case study, only 21 out of 30 available workdays (over three weeks) are required to complete all jobs, indicating nine idle personnel-days. By decreasing the parameter n^R_{kbt} in the MILP model, we evaluate alternative shift scenarios with reduced personnel availability, as shown in Table 11.

Scenario A eliminates idle days from the Base Case without increasing total production time. Scenarios B–I adjust weekly personnel availability and are resolved using the MILP model in less than 10 min each. These adjustments lead to minor increases in production time (10–139 min) due to different sequencing and setup times. However, worker utilization improves significantly, from 66% in the Base Case to over 95%. The analysis shows that at least 8, 6, and 3 personnel-days are needed in weeks one, two, and three, respectively, although this varies slightly by scenario. These scenarios can help the plant's decision makers to accommodate potential personnel time-off requests while meeting the production of all customer orders before their due dates.

7. Conclusion

In this paper, we addressed the UPMSR with realistic manufacturing constraints, including job release/due times, machine eligibility, and resource transition limits. We proposed an MILP model and a BRKGA decoder to minimize the total production time. Computational results show that the BRKGA scales better than the MILP, producing more feasible solutions in the largest instance classes of our benchmark set while maintaining near-optimal quality (average RPD of 1.71%). The advantage pf BRKGA is most pronounced for instances with release/due time constraints, where the MILP often fails to find feasible solutions, highlighting the suitability of BRKGA for large-scale, time-constrained real-world scheduling. Benchmarking on public instances against GRASP, MILP, and Benders-decomposition approaches further confirmed the versatility of the BRKGA, particularly for highly constrained and large-scale RPMSPs. Finally, a case study in a food-processing plant demonstrated the practical value of the MILP model for both production scheduling and personnel shift planning. Future work may extend the BRKGA to RPMSPs with both shared and

Table 11
Possible scenarios for personnel shift schedules.

Scenario	Weekly workload (days)	Available time (min)	Production time (min)	Time diff. from base (min)	Personnel utilization (%)
Base Case	[10, 10, 10]	13,500	8941	–	66%
A	[10, 8, 3]	9450	8941	0	95%
B	[10, 7, 4]	9450	8951	10	95%
C	[10, 6, 5]	9450	9042	101	96%
D	[9, 9, 3]	9450	8956	15	95%
E	[9, 8, 4]	9450	8966	25	95%
F	[9, 7, 5]	9450	8985	44	95%
G	[8, 9, 4]	9450	9061	120	96%
H	[8, 10, 3]	9450	9050	109	96%
I	[8, 8, 5]	9450	9080	139	96%

Table 12
An overview of additional set, and decision variables required in the time-indexed MILP model.

Set	
$\theta \in \Theta = \{0, \ldots, \max_{t \in T} \upsilon_t^R\}$	Set of intra–period time units
Variables	
$H_{ijlbt\theta}$	1 if setup between successive jobs i and j on machine l, bucket b, in period t, ends at instant θ; 0 otherwise.
$U_{ilbt\theta}$	1 if job i is first on machine l, bucket b, in period t, and its setup starts at instant θ; 0 otherwise.
$I_{ihlbt\theta}$	1 if job i is the last processed job on machine l, bucket b, in period t, and setup between successive jobs i and h ends at instant θ; 0 otherwise.
$O^R_{kbt\theta} / L^R_{kbt\theta}$	1 if resource k starts/ends bucket b in period t at instant θ; 0 otherwise.

dedicated resources, which remains underexplored. Another promising direction is to address feasibility detection and infeasibility certification for currently unsolved instances.

CRediT authorship contribution statement

Maziyar Khadivi: Writing – review & editing, Writing – original draft, Visualization, Validation, Methodology, Formal analysis, Data curation, Conceptualization. **Kevin Tierney:** Writing – review & editing, Supervision, Methodology, Conceptualization. **Mostafa Abbasi:** Visualization, Formal analysis, Data curation. **Todd Charter:** Formal analysis, Data curation. **Homayoun Najjaran:** Writing – review & editing, Supervision, Resources, Project administration, Funding acquisition.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used Generative AI tool in order to improve readability and language. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by Natural Sciences and Engineering Research Council (NSERC) of Canada Alliance Grant [ALLRP 555220 - 20]; Mitacs Globalink Research Award [MITACS IT39971]; and NTWIST Inc., Canada. We thank Lopez-Esteve et al. (2023) and Caselli et al. (2024) for sharing their code and we are also grateful to the anonymous referees for their valuable comments and helpful suggestions.

Appendix A

Time-indexed MILP model

Following the time-indexed MILP formulations in Kim and Lee (2021), Yepes-Borrero et al. (2021, 2020), we introduce the set Θ to index discrete time instants within each period and replace all continuous start–finish variables for jobs and buckets with the binary variables listed in Table 12. The objective function (1), the job allocation and sequencing constraints (2)–(6), (10)–(11), and the bucket allocation and sequencing constraints (14)–(20) remain unchanged. The timing constraints for jobs (7)–(9), (12)–(13) and for buckets (21)–(26) are replaced by the following time-indexed constraints.

Time-indexed constraints

$$\sum_{\theta\in\Theta} H_{ijlbt\theta} = Q_{ijlbt} \qquad \forall l\in M,\ b\in\Delta_l,\ t\in T,\ i,j\in N:\ i\neq j \tag{31}$$

$$\sum_{\theta\in\Theta} U_{ilbt\theta} = G_{ilbt} \qquad \forall l\in M,\ b\in\Delta_l,\ t\in T,\ i\in N \tag{32}$$

$$\sum_{\theta\in\Theta} I_{ihlbt\theta} = E_{ihlbt} \qquad \forall l\in M,\ b\in\Delta_l,\ t\in T,\ i\in N,\ h\in N_0:\ h\neq i \tag{33}$$

$$\sum_{\theta\in\Theta} O^R_{kbt\theta} = A^R_{kbt} \qquad \forall k\in K,\ b\in B,\ t\in T \tag{34}$$

$$\sum_{\theta\in\Theta} L^R_{kbt\theta} = A^R_{kbt} \qquad \forall k\in K,\ b\in B,\ t\in T \tag{35}$$

$$\sum_{\theta\in\Theta} \theta\, H_{ijlbt\theta} \geq \sum_{\substack{\ell\in N\\ \ell\neq i}} \sum_{\theta\in\Theta} \big(\theta + p_{il} + s_{ijl}\big) H_{\ell ilbt\theta} - v^R_t\big(1 - Q_{ijlbt}\big) \qquad \forall l\in M,\ b\in\Delta_l,\ t\in T,\ i,j\in N:\ i\neq j \tag{36}$$

$$\sum_{\theta\in\Theta} \theta\, H_{ijlbt\theta} \geq \sum_{\theta\in\Theta} \big(\theta + s^I_{ilbt} + p_{il} + s_{ijl}\big) U_{ilbt\theta} - v^R_t\big(1 - Q_{ijlbt}\big) \qquad \forall l\in M,\ b\in\Delta_l,\ t\in T,\ i,j\in N:\ i\neq j \tag{37}$$

$$\sum_{\theta\in\Theta} \theta\, I_{ihlbt\theta} \geq \sum_{\substack{\ell\in N\\ \ell\neq i}} \sum_{\theta\in\Theta} \big(\theta + p_{il} + s_{ihl}\big) H_{\ell ilbt\theta} - v^R_t\big(1 - E_{ihlbt}\big) \qquad \forall l\in M,\ b\in\Delta_l,\ t\in T,\ i\in N,\ h\in N_0:\ h\neq i \tag{38}$$

$$\sum_{\substack{h\in N_0\\ h\neq i}} \sum_{\theta\in\Theta} \big(\theta - s_{ihl}\big)\, I_{ihlbt\theta} \geq \sum_{\theta\in\Theta} \big(\theta + s^I_{ilbt} + p_{il}\big)\, U_{ilbt\theta} - v^R_t\big(2 - G_{ilbt} - \sum_{\substack{h\in N_0\\ h\neq i}} E_{ihlbt}\big) \qquad \forall l\in M,\ b\in\Delta_l,\ t\in T,\ i\in N \tag{39}$$

$$\sum_{k\in K}\sum_{\theta\in\Theta} \theta\, O^R_{kbt\theta} \leq \sum_{i\in N}\sum_{\theta\in\Theta} \theta\, U_{ilbt\theta} + v^R_t\big(1 - \sum_{i\in N} G_{ilbt}\big) \qquad \forall l\in M,\ b\in\Delta_l,\ t\in T \tag{40}$$

$$\sum_{k\in K}\sum_{\theta\in\Theta} \theta\, L^R_{kbt\theta} \geq \sum_{i\in N}\sum_{\substack{h\in N_0\\ h\neq i}}\sum_{\theta\in\Theta} \theta\, I_{ihlbt\theta} - v^R_t\big(1 - \sum_{i\in N}\sum_{\substack{h\in N_0\\ h\neq i}} E_{ihlbt}\big)$$

$$\sum_{k\in K}\sum_{\theta\in\Theta} \theta\, L^R_{kbt\theta} \geq \sum_{k\in K}\sum_{\theta\in\Theta} \theta\, O^R_{kbt\theta} + \sum_{i\in N} p_{il} A_{ilbt} + \sum_{\substack{i,j\in N\\ i\neq j}} s_{ijl} Q_{ijlbt} + \sum_{i\in N} s^I_{ilbt} G_{ilbt} + \sum_{i\in N}\sum_{\substack{h\in N_0\\ h\neq i}} s_{ihl} E_{ihlbt} \qquad \forall l\in M,\ b\in\Delta_l,\ t\in T \tag{41}$$

$$\sum_{\theta\in\Theta} \theta\, O^R_{kut\theta} \geq \sum_{\theta\in\Theta} \theta\, L^R_{kbt\theta} + v^R_t\big(Q^R_{kbut} - A^R_{kbt}\big) \qquad \forall k\in K,\ t\in T,\ b,u\in B:\ b\neq u \tag{42}$$

$$\sum_{k\in K}\sum_{\theta\in\Theta} \theta\, O^R_{k\,b+1\,t\theta} \geq \sum_{k\in K}\sum_{\theta\in\Theta} \theta\, L^R_{kbt\theta} + v^R_t\big(\sum_{k\in K} A^R_{k\,b+1\,t} - \sum_{k\in K} A^R_{kbt}\big) \qquad \forall l\in M,\ b,b+1\in\Delta_l,\ t\in T \tag{43}$$

$$\sum_{\substack{j\in N\\ j\neq i}}\sum_{\theta\in\Theta} \big(\theta - p_{il} - s_{ijl}\big)\, H_{ijlbt\theta} \geq r_{it} - v^R_t\big(1 - \sum_{\substack{j\in N\\ j\neq i}} Q_{ijlbt}\big) \qquad \forall l\in M,\ b\in\Delta_l,\ t\in T,\ i\in N \tag{44}$$

$$\sum_{\theta\in\Theta} \big(\theta + s^I_{ilbt}\big)\, U_{ilbt\theta} \geq r_{it} - v^R_t\big(1 - G_{ilbt}\big) \qquad \forall l\in M,\ b\in\Delta_l,\ t\in T,\ i\in N \tag{45}$$

$$\sum_{\substack{h\in N_0\\ h\neq i}}\sum_{\theta\in\Theta} \big(\theta - p_{il} - s_{ihl}\big)\, I_{ihlbt\theta} \geq r_{it} - v^R_t\big(1 - \sum_{\substack{h\in N_0\\ h\neq i}} E_{ihlbt}\big) \qquad \forall l\in M,\ b\in\Delta_l,\ t\in T,\ i\in N \tag{46}$$

$$\sum_{\theta\in\Theta} \big(\theta + s^I_{ilbt} + p_{il}\big)\, U_{ilbt\theta} \leq d_{it} + v^R_t\big(1 - A_{ilbt}\big) \qquad \forall l\in M,\ b\in\Delta_l,\ t\in T,\ i\in N \tag{47}$$

$$\sum_{\substack{j\in N\\ j\neq i}}\sum_{\theta\in\Theta} \big(\theta - s_{ijl}\big)\, H_{ijlbt\theta} \leq d_{it} + v^R_t\big(1 - A_{ilbt}\big) \qquad \forall l\in M,\ b\in\Delta_l,\ t\in T,\ i\in N \tag{48}$$

$$\sum_{\substack{h\in N_0\\ h\neq i}}\sum_{\theta\in\Theta} \big(\theta - s_{ihl}\big)\, L_{ihlbt\theta} \leq d_{it} + v^R_t\big(1 - A_{ilbt}\big) \qquad \forall l\in M,\ b\in\Delta_l,\ t\in T,\ i\in N \tag{49}$$

$$\qquad\qquad\qquad

Table 13
Comparison of time-continuous and time-indexed MILP formulations.

Metric	Without time windows		With time windows	
	Time-continuous	Time-indexed	Time-continuous	Time-indexed
Binary variables	256	504,492	256	504,492
Continuous variables	64	0	64	0
Constraints	358	1601	368	1721
Total production time (min)	1527	1527	1693	1693
CPU time (s)	0.03	174.26	0.02	28.53

constraints (43) coordinate the timing of each resource across the buckets to which it is allocated, while constraints (44) require buckets on each machine to be used in ascending temporal order. Finally, constraints (45)–(50) ensure that every job is processed within its release and due time.

Computational comparison of time-continuous and time-indexed MILP models

To compare the time-continuous MILP model developed in Section 4 with the time-indexed model proposed here, we benchmark both formulations on the toy example described in Section 3. As shown in the table, both models yield the same total production time; however, the time-indexed model requires noticeably longer solution times, particularly when production time windows are removed, as these windows act as feasibility cuts that reduce the search space. The higher computational burden of the time-indexed formulation stems from its substantially larger number of variables and constraints introduced by the discrete time index θ, rather than using continuous timing variables. This behavior is consistent with prior findings, (see Fanjul-Peyro, 2020; Floudas & Lin, 2004). Moreover, when applied to the smallest real problem instance in our study (30 jobs and 2 machines), the time-indexed model was unable even to solve the LP relaxation (i.e., lower bound) within five minutes, let alone produce a feasible solution. Consequently, we report results only for the toy example (see Table 13).

Illustrative example of the MILP model

The MILP model described in Section 4 is illustrated in Fig. 5 using the toy example introduced in Section 3.1. The model comprises jobs, machine buckets, and resources. Each machine is assigned two buckets, ensuring that at most one resource transition can occur per machine within each time period (in this example, a single week). The decision process involves the allocation, sequencing, and timing of jobs and resources on machine buckets. For illustrative purposes, these decisions are decomposed into three blocks in Fig. 5; however, in the actual model they are determined simultaneously, as allocation and sequencing decisions directly influence timing.

First, the allocation of jobs and resources to buckets is determined by Constraints (2), (14), and (15). Next, job sequencing within each machine bucket is enforced through Constraints (3)–(6). For example, on bucket 4 of machine 2, job 3 is selected as the first job, indicated by $G^J_{i=3,l=2,b=4,t=1}=1$, and job 5 is the last job, indicated by $E^J_{i=5,l=2,b=4,t=1}=1$. The internal job sequence is captured by $Q^J_{i=3,j=6,l=2,b=4,t=1}=1$ and $Q^J_{i=6,j=5,l=2,b=4,t=1}=1$. Similarly, the sequencing of resource operations across buckets is determined by Constraints (16) and (19). For instance, resource 1 starts operating on bucket 1 in period 1, i.e., $G^R_{k=1,b=1,t=1}=1$, and finishes on bucket 4, i.e., $E^R_{k=1,b=4,t=1}=1$. Its movement across buckets is represented by the sequence variables $Q^R_{k=1,b=1,u=3,t=1}=1$ (from B1 to B3), $Q^R_{k=1,b=3,u=2,t=1}=1$ (from B3 to B2), and $Q^R_{k=1,b=2,u=4,t=1}=1$ (from B2 to B4).

The timing decisions for jobs and buckets are illustrated in the final block of Fig. 5. Job start and completion times are governed by Constraints (7)–(9), while release and due-date requirements are enforced through Constraints (10)–(13). The timing of buckets, accounting for both their sequencing and the timing of the jobs they contain, is determined by Constraints (20)–(28). To ensure that sequence-dependent setups between consecutive buckets are respected within each time period, the setup between the last job of bucket b and the first job of bucket $b+1$ is enforced by Constraint (29). For example, this captures the setup between the ending job J2 of bucket B3 and the first job J3 of bucket B4 in Fig. 5. Finally, continuity across time periods is maintained by Constraint (30), which enforces connectivity between the last bucket of a machine in period t and the first bucket of the same machine in period $t+1$. This mechanism is not illustrated in Fig. 5, as the figure depicts a single time period.

Comparison of single-step and two-step MILP solution approaches

As shown in Table 14, the proposed two-step solution approach consistently yields an equal or larger number of feasible solutions compared to the single-step MILP formulation that directly minimizes total production time. The only exception occurs for instance class $(120, 2, 1, 1)$, where the two-step approach produces 6 feasible solutions versus 7 obtained by the single-step model. Overall, the two-step model finds 109 feasible solutions out of 171 instances, whereas the single-step formulation achieves feasibility for only 91 instances.

Both approaches are executed under 30-minute time limit. The two-step strategy incurs a substantially higher computational cost, with an average solution time of 15.28 min compared to 8.35 min for the single-step formulation. This increase is expected, since the two-step framework first uses the available time budget to solve the first-step problem, maximizing the number of scheduled jobs and identifying a feasible warm-start solution, before using the remaining time to refine this solution by minimizing total production time in the second step.

Regarding solution quality, both approaches exhibit comparable optimality performance. The total number of instances solved to proven optimality is similar (72 for single-step and 68 for two-step), and the average optimality gaps remain low for both formulations (0.14% and 1.78%, respectively). These results indicate that the two-step approach improves feasibility coverage at the expense of increased computational effort, while maintaining competitive optimality performance.

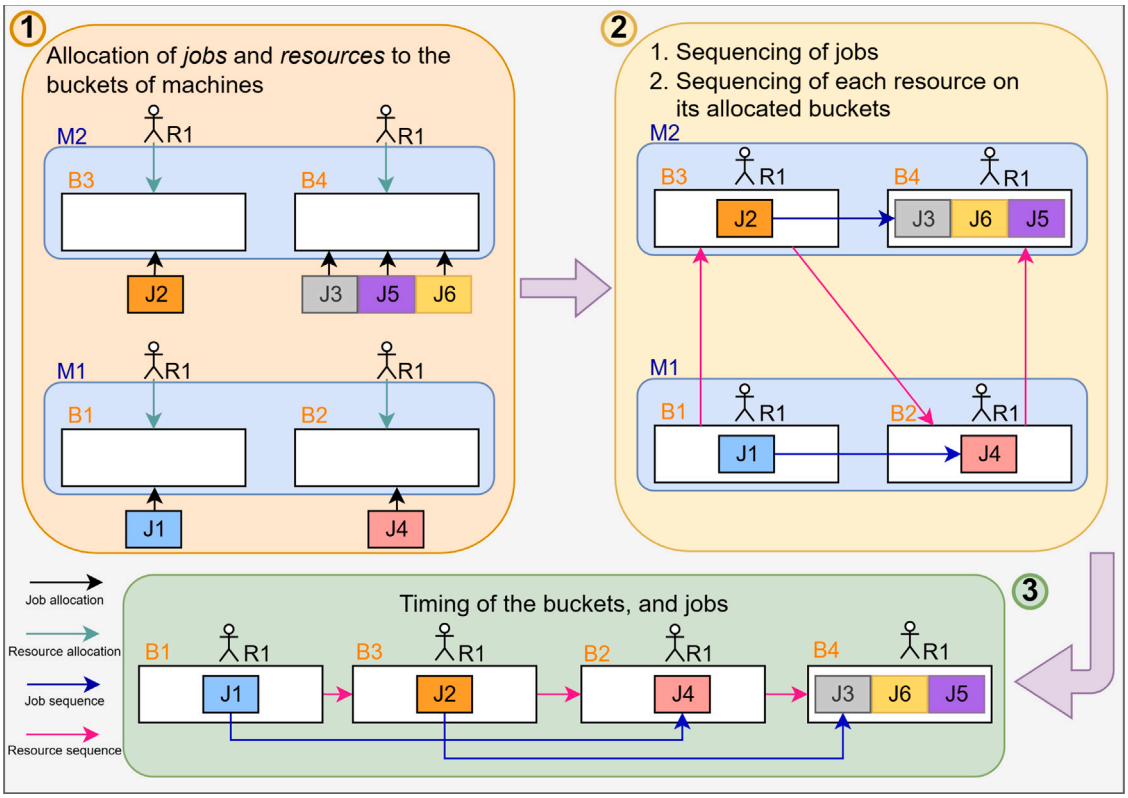


Fig. 5. Illustration of the MILP Model on the toy example.

Table 14
Single-step vs. two-step MILP performance by instance class.

Instances	#	Single-step				Two-step			
(J, M, P, W)		# Feasible (Infeasible)	# Optimal solutions	Total time (min)	Avg gap (%)	# Feasible (Infeasible)	# Optimal solutions	Total time (min)	Avg gap (%)
(30, 2, 1, 1)	9	9	9	0.03	0.00	9	9	0.07	0.00
(60, 2, 1, 1)	9	8 (1)	8	1.08	0.00	8 (1)	7	4.59	0.12
(60, 4, 3, 1)	9	9	5	10.42	0.17	9	6	11.09	0.32
(90, 2, 1, 1)	9	8 (1)	8	3.19	0.00	8 (1)	7	5.46	0.07
(90, 2, 1, 3)	9	9	5	18.73	0.21	9	4	24.16	0.73
(90, 4, 3, 1)	9	6	4	11.99	0.10	8	5	16.82	0.71
(120, 2, 1, 1)	9	7 (2)	6	5.52	0.05	6 (1)	6	4.14	0.00
(120, 2, 1, 3)	9	7	2	25.40	1.15	8	0	30.04	1.74
(120, 4, 3, 1)	9	4	3	8.06	0.08	8	3	21.03	2.17
(120, 8, 3, 1)	9	3	3	1.34	0.00	6	3	18.21	2.15
(120, 8, 6, 1)	9	5	3	12.77	0.05	6	3	18.09	0.34
(180, 2, 1, 3)	9	1	1	6.81	0.00	2	0	30.31	0.25
(180, 2, 1, 6)	9	0	0	–	–	1	0	30.06	2.45
(180, 4, 3, 1)	9	3	3	0.75	0.00	5	3	14.75	1.17
(180, 4, 3, 3)	9	0	0	–	–	2	0	30.06	51.82
(180, 8, 3, 1)	9	3	3	5.37	0.00	3	3	13.14	0.00
(180, 8, 6, 1)	9	3	3	6.74	0.00	4	3	18.04	1.38
(180, 10, 6, 1)	9	3	3	3.72	0.00	4	3	18.51	3.19
(180, 10, 9, 1)	9	3	3	6.09	0.00	3	3	15.09	0.00
Total/Avg.:	**171**	**91 (4)**	**72**	**8.35**	**0.14**	**109 (3)**	**68**	**15.04**	**1.78**

BRKGA decoder alternatives

The idea behind the design of the BRKGA decoder is to ensure feasibility with respect to the problem constraints, either by strictly enforcing them during schedule construction or by penalizing violations in the fitness function. The decoder presented in Section 5 schedules jobs by allowing processing to start only after their release times, which are therefore strictly respected. In contrast, due dates are handled indirectly by penalizing total tardiness in the objective function. Penalizing job earliness is not suitable in our setting, as the decoder assigns jobs greedily based on the next available machine time and does not explicitly decide how much idle time to insert before processing a job. Consequently, penalizing earliness does not guarantee satisfaction of release-time constraints. While a repair procedure could be applied after constructing a complete schedule to shift jobs forward in time, doing so for every individual in each generation would be computationally expensive and could significantly decrease

Table 15
Comparison of base and alternative decoder architectures.

Metric	Base decoder	Alt. I	Alt. II
Feasible solutions (out of 171)	158	133	130
Average RPD (%)	–	0.12	0.04

convergence speed. Moreover, repairing only the final solution to eliminate release-time violations may introduce due time violations by shifting jobs forward. For these reasons, we enforce release times strictly by scheduling each job no earlier than its release time. Due time constraints cannot be strictly enforced during schedule construction, as satisfying them may require swapping jobs with earlier scheduled ones, potentially increasing production time and introducing new violations for previously scheduled jobs. We therefore allow the decoder to learn suitable job orderings by penalizing tardiness rather than enforcing due times explicitly.

With respect to resource constraints, prior studies (e.g., Lopez-Esteve et al., 2023; Yepes-Borrero et al., 2020) apply a posterior repair phase to resolve violations caused by parallel job execution. However, such approaches are not suitable for our problem due to the presence of resource transition limits. Post-processing repairs do not guarantee compliance with transition constraints and may also lead to due-time violations when jobs are shifted forward. We therefore enforce resource feasibility directly during decoding. Regarding resource transition limit, since the timing of machine runs subject to this limit cannot be determined *a priori*, violations are penalized within the fitness function, allowing the decoder to learn appropriate decisions on the frequency and timing of machine operations.

Based on the above rationale, we investigate two alternative decoder architectures. Both variants enforce release-time and resource-allocation constraints during decoding, while handling resource transition violations and job tardiness through penalization, similarly to the decoder described in Section 5. However, their internal structures differ from that of the baseline decoder. In the first alternative (Alt. I), the decoder does not iterate over candidate machines in Lines 16–26 of Algorithm 2 to identify a machine that avoids resource transition violations. Instead, it immediately selects the first available resource. Accordingly, the while loop and fallback mechanism (Lines 15–32 of Algorithm 2) are omitted and replaced by the modified procedure presented in Algorithm 4.

Algorithm 4 Decoder heuristic modification for Alternative I

$M^* \leftarrow \arg\min_{l \in \gamma_j} (T_l,\ C'_l)$
$R^*,\ CR'_{R^*} \leftarrow$ **ResourceAssign**$(CR,\ C_{M^*},\ S_{M^*},\ P_{M^*})$
Find current time period (week): $t \leftarrow \left\lfloor \frac{\max(C_{M^*},\ CR'_{R^*})}{\beta} \right\rfloor$
if resource R^* is assigned to a different machine than its last assignment **then**
    $WT_{M^*,t} \leftarrow WT_{M^*,t} + 1$
    if $WT_{M^*,t} \geq \delta$ **then**
        $TN \leftarrow TN + 1$

In the second alternative (Alt. II), rather than looping over remaining machines to identify one that satisfies the transition limit, the decoder selects the machine that minimizes the production and completion time of the current job j. It then iterates over the set of remaining resources RR to identify a feasible resource for the selected machine. This modification is implemented by replacing the variable RM in Algorithm 2 with RR and substituting Lines 15–26 with the procedure described in Algorithm 5.

Algorithm 5 Decoder heuristic modification for Alternative II

$M^* \leftarrow \arg\min_{l \in \gamma_j} (T_l,\ C'_l)$
$RR \leftarrow ResourceList[1, \ldots, R]$
while $RR \neq \emptyset$ **do**
    $R^*,\ CR'_{R^*} \leftarrow$ **ResourceAssign**$(CR,\ C_{M^*},\ S_{M^*},\ P_{M^*})$
    Find current time period (week): $t \leftarrow \left\lfloor \frac{\max(C_{M^*},\ CR'_{R^*})}{\beta} \right\rfloor$
    if resource R^* is assigned to a different machine than its last assignment **then**
        $WT_{M^*,t} \leftarrow WT_{M^*,t} + 1$
        if $WT_{M^*,t} \geq \delta$ **then** ▷ Check if transition limit exceeded
            Remove R^* from the remaining resource set: $RR \leftarrow RR \setminus \{R^*\}$
            Undo the increment: $WT_{M^*,t} \leftarrow WT_{M^*,t} - 1$
        else break ▷ Accept R^* and stop evaluating other resources
    else break ▷ No transition; accept R^*

We evaluate the BRKGA using the base decoder and the two alternative variants under identical experimental settings and instance sets described in Section 6. Each experiment consists of a single BRKGA run with a time limit of 2×the number of jobs, where 20 independent populations are evolved in parallel, each on a separate CPU thread with a different random seed. As reported in Table 15, the base decoder produces substantially more feasible solutions than either alternative. All infeasible solutions generated by Alt. I and Alt. II are due to violations of the resource transition constraint. This result highlights the importance of explicitly searching over all eligible machines for the current job, rather than relying on greedy machine selection or searching over available resources. In terms of solution quality, when feasible solutions are obtained, both alternative decoders achieve performance comparable to the base decoder, with average RPD values close to zero.

Appendix B. Supplementary data

Supplementary material related to this article can be found online at https://doi.org/10.1016/j.ejor.2026.05.025.

References

Afzalirad, M., & Rezaeian, J. (2016). Resource-constrained unrelated parallel machine scheduling problem with sequence dependent setup times, precedence constraints and machine eligibility restrictions. *Computers & Industrial Engineering*, *98*, 40–52.

AIMMS 24. 5. 5. 4 software (2025), https://www.aimms.com/.

Akyol Ozer, E., & Sarac, T. (2019). MIP models and a matheuristic algorithm for an identical parallel machine scheduling problem under multiple copies of shared resources constraints. *Top*, *27*, 94–124.

Al-Harkan, I. M., & Qamhan, A. A. (2019). Optimize unrelated parallel machines scheduling problems with multiple limited additional resources, sequence-dependent setup times and release date constraints. *IEEE Access*, *7*, 171533–171547.

Al-Harkan, I. M., Qamhan, A. A., Badwelan, A., Alsamhan, A., & Hidri, L. (2021). Modified harmony search algorithm for resource-constrained parallel machine scheduling problem with release dates and sequence-dependent setup times. *Processes*, *9*(4), 654.

Ansótegui, C., Sellmann, M., & Tierney, K. (2009). A gender-based genetic algorithm for the automatic configuration of algorithms. In *International conference on principles and practice of constraint programming* (pp. 142–157). Springer.

Avgerinos, I., Mourtos, I., Vatikiotis, S., & Zois, G. (2023). Scheduling unrelated machines with job splitting, setup resources and sequence dependency. *International Journal of Production Research*, *61*(16), 5502–5524.

Barták, R., & Čepek, O. (2006). A constraint model for state transitions in disjunctive resources. In *International workshop on constraint solving and constraint logic programming* (pp. 48–62). Springer.

Basso, F., & Varas, M. (2017). A MIP formulation and a heuristic solution approach for the bottling scheduling problem in the wine industry. *Computers & Industrial Engineering*, *105*, 136–145.

Bektur, G., & Saraç, T. (2019). A mathematical model and heuristic algorithms for an unrelated parallel machine scheduling problem with sequence-dependent setup times, machine eligibility restrictions and a common server. *Computers & Operations Research*, *103*, 46–63.

Bitar, A., Dauzère-Pérès, S., & Yugma, C. (2021). Unrelated parallel machine scheduling with new criteria: Complexity and models. *Computers & Operations Research*, *132*, Article 105291.

Bitar, A., Dauzère-Pérès, S., Yugma, C., & Roussel, R. (2016). A memetic algorithm to solve an unrelated parallel machine scheduling problem with auxiliary resources in semiconductor manufacturing. *Journal of Scheduling*, *19*(4), 367–376.

Caricato, P., Grieco, A., Arigliano, A., & Rondone, L. (2021). Workforce influence on manufacturing machines schedules. *International Journal of Advanced Manufacturing Technology*, *115*, 915–925.

Caselli, G., Delorme, M., Iori, M., & Magni, C. A. (2024). Exact algorithms for a parallel machine scheduling problem with workforce and contiguity constraints. *Computers & Operations Research*, *163*, Article 106484.

Celano, G., Costa, A., & Fichera, S. (2008). Scheduling of unrelated parallel manufacturing cells with limited human resources. *International Journal of Production Research*, *46*(2), 405–427.

Chen, J.-F. (2005). Unrelated parallel machine scheduling with secondary resource constraints. *International Journal of Advanced Manufacturing Technology*, *26*, 285–292.

Chen, H., Guo, P., Jimenez, J., Dong, Z. S., & Cheng, W. (2022). Unrelated parallel machine photolithography scheduling problem with dual resource constraints. *IEEE Transactions on Semiconductor Manufacturing*, *36*(1), 100–112.

Chen, J., Li, K., Chu, C., & Sahli, A. (2024). A simplified swarm optimization algorithm to minimize makespan on non-identical parallel machines with unequal job release times under non-renewable resource constraints. *Operational Research*, *24*(2), 21.

Conover, W. J. (1999). *Practical nonparametric statistics*. john wiley & sons.

Costa, A., Cappadonna, F., & Fichera, S. (2013). A hybrid genetic algorithm for job sequencing and worker allocation in parallel unrelated machines with sequence-dependent setup times. *International Journal of Advanced Manufacturing Technology*, *69*, 2799–2817.

Dai, F., & Lei, D. (2023). An imperialist competitive algorithm with adaptive cooperation for resource-constrained unrelated parallel machines scheduling with setup time. In *2023 35th Chinese control and decision conference* (pp. 5128–5133). IEEE.

Edis, E. B., Oguz, C., & Ozkarahan, I. (2013). Parallel machine scheduling with additional resources: Notation, classification, models and solution methods. *European Journal of Operational Research*, *230*(3), 449–463.

Fang, K., Wang, S., Pinedo, M. L., Chen, L., & Chu, F. (2021). A combinatorial benders decomposition algorithm for parallel machine scheduling with working-time restrictions. *European Journal of Operational Research*, *291*(1), 128–146.

Fanjul-Peyro, L. (2020). Models and an exact method for the unrelated parallel machine scheduling problem with setups and resources. *Expert Systems with Applications: X*, *5*, Article 100022.

Fanjul-Peyro, L., Perea, F., & Ruiz, R. (2017). Models and matheuristics for the unrelated parallel machine scheduling problem with additional resources. *European Journal of Operational Research*, *260*(2), 482–493.

Fanjul-Peyro, L., Ruiz, R., & Perea, F. (2019). Reformulations and an exact algorithm for unrelated parallel machine scheduling problems with setup times. *Computers & Operations Research*, *101*, 173–182.

Fleszar, K., & Hindi, K. S. (2018). Algorithms for the unrelated parallel machine scheduling problem with a resource constraint. *European Journal of Operational Research*, *271*(3), 839–848.

Floudas, C. A., & Lin, X. (2004). Continuous-time versus discrete-time approaches for scheduling of chemical processes: a review. *Computers & Chemical Engineering*, *28*(11), 2109–2129.

Gao, M., Liu, C., & Chen, X. (2025). A bi-objective unrelated parallel machine scheduling problem with additional resources and soft precedence constraints. *European Journal of Operational Research*.

Gonçalves, J. F., & Resende, M. G. (2011). Biased random-key genetic algorithms for combinatorial optimization. *Journal of Heuristics*, *17*(5), 487–525.

Gurobi 12. 0. 1 solver (2025), https://www.gurobi.com/.

Ham, A. (2017). Scheduling of dual resource constrained lithography production: Using CP and MIP/CP. *IEEE Transactions on Semiconductor Manufacturing*, *31*(1), 52–61.

Heinz, V., Novák, A., Vlk, M., & Hanzálek, Z. (2022). Constraint programming and constructive heuristics for parallel machine scheduling with sequence-dependent setups and common servers. *Computers & Industrial Engineering*, *172*, Article 108586.

Hottung, A., & Tierney, K. (2016). A biased random-key genetic algorithm for the container pre-marshalling problem. *Computers & Operations Research*, *75*, 83–102.

Kemminer, R., Lange, J., Kempkes, J. P., Tierney, K., & Weiß, D. (2024). Configuring mixed-integer programming solvers for large-scale instances. *Operations Research Forum*, *5*(2), 48.

Kim, H.-J., & Lee, J.-H. (2021). Scheduling uniform parallel dedicated machines with job splitting, sequence-dependent setup times, and multiple servers. *Computers & Operations Research*, *126*, Article 105115.

Lei, D., & He, S. (2022). An adaptive artificial bee colony for unrelated parallel machine scheduling with additional resource and maintenance. *Expert Systems with Applications*, *205*, Article 117577.

Li, K., Xu, L., Zhang, H., & Chen, J. (2024). A genetic algorithm for fuzzy identical parallel machine scheduling of minimising total weighted tardiness under resource constraint. *International Journal of Production Research*, 1–25.

Liang, Y., Sun, Z., Song, T., Chou, Q., Fan, W., Fan, J., Rui, Y., Zhou, Q., Bai, J., Yang, C., et al. (2022). Lenovo schedules laptop manufacturing using deep reinforcement learning. *INFORMS Journal on Applied Analytics*, *52*(1), 56–68.

Lopez-Esteve, A., Perea, F., & Yepes-Borrero, J. C. (2023). GRASP algorithms for the unrelated parallel machines scheduling problem with additional resources during processing and setups. *International Journal of Production Research*, *61*(17), 6013–6029.

Mitchell, M. (1998). *An introduction to genetic algorithms*. MIT Press.
Munoz, L., Villalobos, J. R., & Fowler, J. W. (2022). Exact and heuristic algorithms for the parallel machine total completion time scheduling problem with dual resources, ready times, and sequence-dependent setup times. *Computers & Operations Research*, *143*, Article 105787.
Raboudi, H., Alpan, G., Mangione, F., Tissot, G., & Frédéric, N. (2022). Scheduling unrelated parallel machines with a common server and sequence dependent setup times. *IFAC-PapersOnLine*, *55*(10), 2179–2184.
Ruiz-Torres, A. J., López, F. J., & Ho, J. C. (2007). Scheduling uniform parallel machines subject to a secondary resource to minimize the number of tardy jobs. *European Journal of Operational Research*, *179*(2), 302–315.
Shafiee, M., Amiri-Aref, M., & Klibi, W. (2025). The integration of shared renewable resources considering setup times for the parallel machine scheduling problem. *Computers & Industrial Engineering*, *200*, Article 110828.
Soares, L. C., & Carvalho, M. A. (2022). Application of a hybrid evolutionary algorithm to resource-constrained parallel machine scheduling with setup times. *Computers & Operations Research*, *139*, Article 105637.
Tahmassebi, T. (1999). Vehicle routing problem (VRP) formulation for continuous-time packing hall design/operations. *Computers & Chemical Engineering*, *23*, S1011–S1014.
Tamaki, H., Hasegawa, Y., Kozasa, J., & Araki, M. (1993). Application of search methods to scheduling problem in plastics forming plant: A binary representation approach. In *Proceedings of 32nd IEEE conference on decision and control* (pp. 3845–3850). IEEE.
Vallada, E., Villa, F., & Fanjul-Peyro, L. (2019). Enriched metaheuristics for the resource constrained unrelated parallel machine scheduling problem. *Computers & Operations Research*, *111*, 415–424.
Villa, F., Vallada, E., & Fanjul-Peyro, L. (2018). Heuristic algorithms for the unrelated parallel machine scheduling problem with one scarce additional resource. *Expert Systems with Applications*, *93*, 28–38.
Yepes-Borrero, J. C., Perea, F., Ruiz, R., & Villa, F. (2021). Bi-objective parallel machine scheduling with additional resources during setups. *European Journal of Operational Research*, *292*(2), 443–455.
Yepes-Borrero, J. C., Villa, F., Perea, F., & Caballero-Villalobos, J. P. (2020). GRASP algorithm for the unrelated parallel machine scheduling problem with setup times and additional resources. *Expert Systems with Applications*, *141*, Article 112959.
Yunusoglu, P., & Topaloglu Yildiz, S. (2022). Constraint programming approach for multi-resource-constrained unrelated parallel machine scheduling problem with sequence-dependent setup times. *International Journal of Production Research*, *60*(7), 2212–2229.
Zhang, L., Deng, Q., Lin, R., Gong, G., & Han, W. (2021). A combinatorial evolutionary algorithm for unrelated parallel machine scheduling problem with sequence and machine-dependent setup times, limited worker resources and learning effect. *Expert Systems with Applications*, *175*, Article 114843.